AWS VPC Hands-On Lab

Complete Beginner's Step-by-Step Guide

Build a Custom VPC with Public & Private Subnets,
Internet Gateway, Route Tables, Security Groups,
EC2 Instances, Bastion Host & NAT Gateway

Region: us-east-1 (N. Virginia)

Date: October 21, 2026

Prepared by: Sahil Kumar

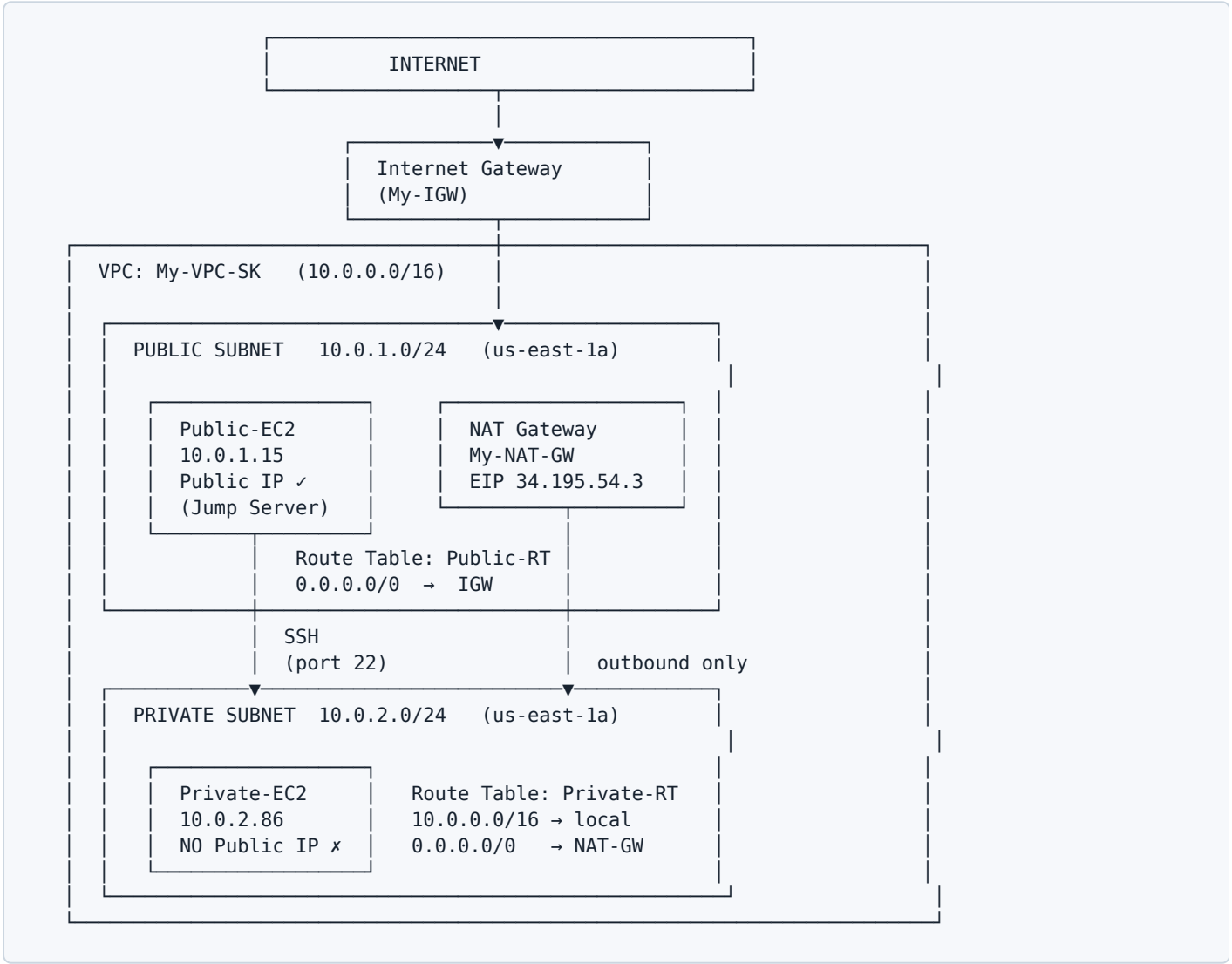
1. Lab Overview & Architecture

1.1 What You Will Build

In this lab you will build a complete, production-style AWS network from scratch. You will create an isolated virtual network (VPC), split it into a public and a private half, launch one server in each half, and then prove three things with real tests:

1. A server in the **public subnet** can be reached from the internet.
2. A server in the **private subnet** cannot be reached from the internet, but *can* be reached from the public server (this is the **Bastion / Jump Server** pattern).
3. A server in the private subnet cannot browse the internet until you add a **NAT Gateway** — and then it can.

1.2 Architecture Diagram



1.3 Resources You Will Create

Resource	Name	Configuration
VPC	My-VPC-SK	CIDR `10.0.0.0/16` — 65,536 IP addresses
Public Subnet	Public-Subnet	`10.0.1.0/24`, AZ us-east-1a, auto-assign public IP ON
Private Subnet	Private-Subnet	`10.0.2.0/24`, AZ us-east-1a, auto-assign public IP OFF
Internet Gateway	My-IGW	Attached to My-VPC-SK
Route Table	Public-RT	`0.0.0.0/0 → IGW`, associated with Public-Subnet
Route Table	Private-RT	`0.0.0.0/0 → NAT-GW`, associated with Private-Subnet
Security Group	My-SG	Inbound: SSH 22, HTTP 80 · Outbound: All traffic
EC2 Instance	Public-EC2	Amazon Linux 2023, t3.micro, public IP enabled
EC2 Instance	Private-EC2	Amazon Linux 2023, t3.micro, no public IP
NAT Gateway	My-NAT-GW	In Public-Subnet, with Elastic IP

1.4 Prerequisites

- An AWS account with permission to create VPC and EC2 resources.
- An EC2 Key Pair (this lab uses one named `test`). Download the `test.pem` file and keep it safe.
- A terminal (Linux / macOS / Windows PowerShell or Git Bash) to run SSH commands.
- Region set to **US East (N. Virginia) — us-east-1** in the top-right corner of the AWS Console.

> **Cost Warning**
> The NAT Gateway and the Elastic IP are **not free-tier eligible**. A NAT Gateway costs roughly $0.045 per hour plus data processing charges. **Delete the NAT Gateway and release the Elastic IP as soon as you finish the lab** to avoid unexpected charges. Cleanup instructions are in Section 18.

2. Step 1 — Create the VPC

A **VPC (Virtual Private Cloud)** is your own isolated section of the AWS cloud. Think of it as a private data centre: you choose the IP address range, and nothing from outside can enter unless you explicitly allow it.

2.1 Navigate to the VPC Console

Click path: In the AWS Console search bar at the top, type `VPC` → click **VPC** in the results → in the left sidebar click **Your VPCs** → click the orange **Create VPC** button.

2.2 Fill in the VPC Settings

Field	What to select / type
Resources to create	Select **VPC only** (not "VPC and more")
Name tag	Type `My-VPC-SK`
IPv4 CIDR block	Select **IPv4 CIDR manual input**, then type `10.0.0.0/16`
IPv6 CIDR block	Leave as **No IPv6 CIDR block**
Tenancy	Leave as **Default**

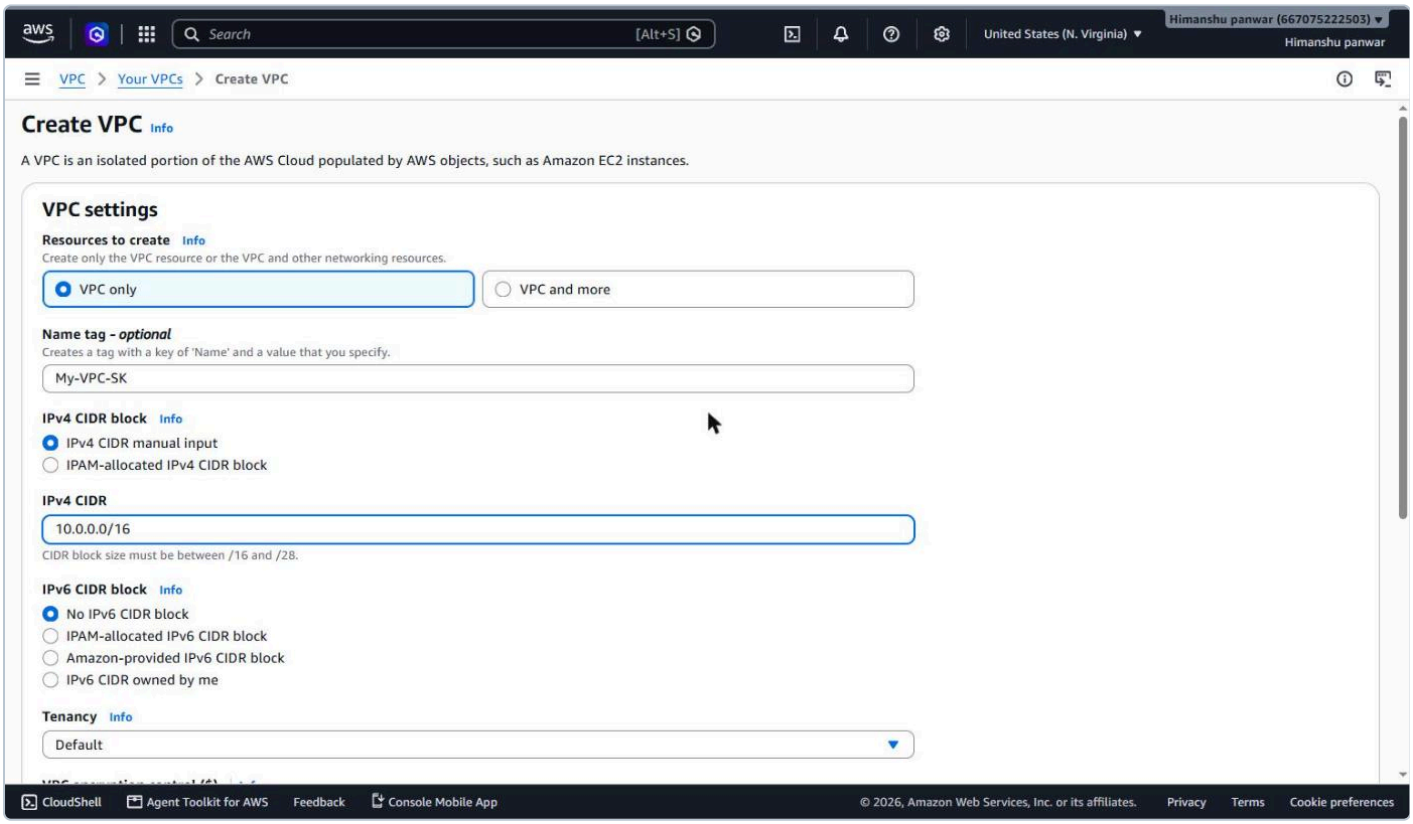


Figure 2.1 — The Create VPC form. Note "VPC only" is selected and the CIDR is 10.0.0.0/16.

Why 10.0.0.0/16?

The `/16` means the first 16 bits are fixed, leaving 16 bits for hosts — that is 65,536 addresses from 10.0.0.0 to 10.0.255.255. This gives you plenty of room to carve out many `/24` subnets (256 addresses each) later. `10.x.x.x` is a private range defined by RFC 1918, so it never conflicts with public internet addresses.

2.3 Create and Verify

Click path: Scroll to the bottom → click **Create VPC**.

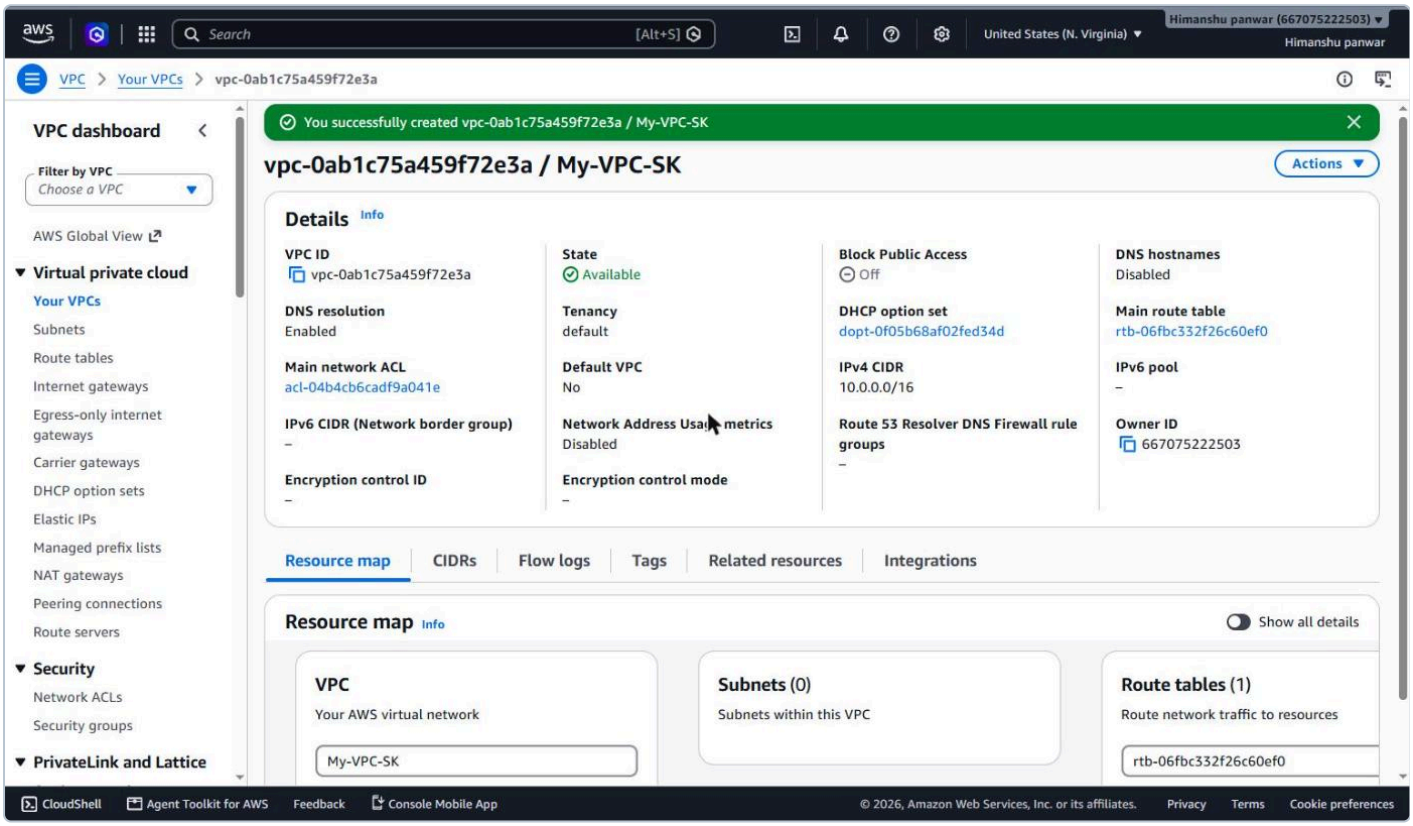


Figure 2.2 — VPC created. The green banner confirms success and the State shows "Available".

Checkpoint

You should now see **My-VPC-SK** in the VPC list with State = **Available**. Write down the VPC ID (it looks like `vpc-0ab1c75a459f72e3a`) — you will select it repeatedly in the next steps.

3. Step 2 — Create the Public Subnet

A **subnet** is a slice of your VPC's IP range, pinned to a single Availability Zone. A subnet becomes "public" only when its route table has a route to an Internet Gateway — there is no checkbox called "public". We create the subnet now and make it public in Step 5.

3.1 Create the Subnet

Click path: In the VPC console left sidebar click **Subnets** → click the orange **Create subnet** button.

Field	What to select / type
VPC ID	Select **My-VPC-SK** from the dropdown
Subnet name	Type `Public-Subnet`
Availability Zone	Select **us-east-1a**
IPv4 subnet CIDR block	Type `10.0.1.0/24`

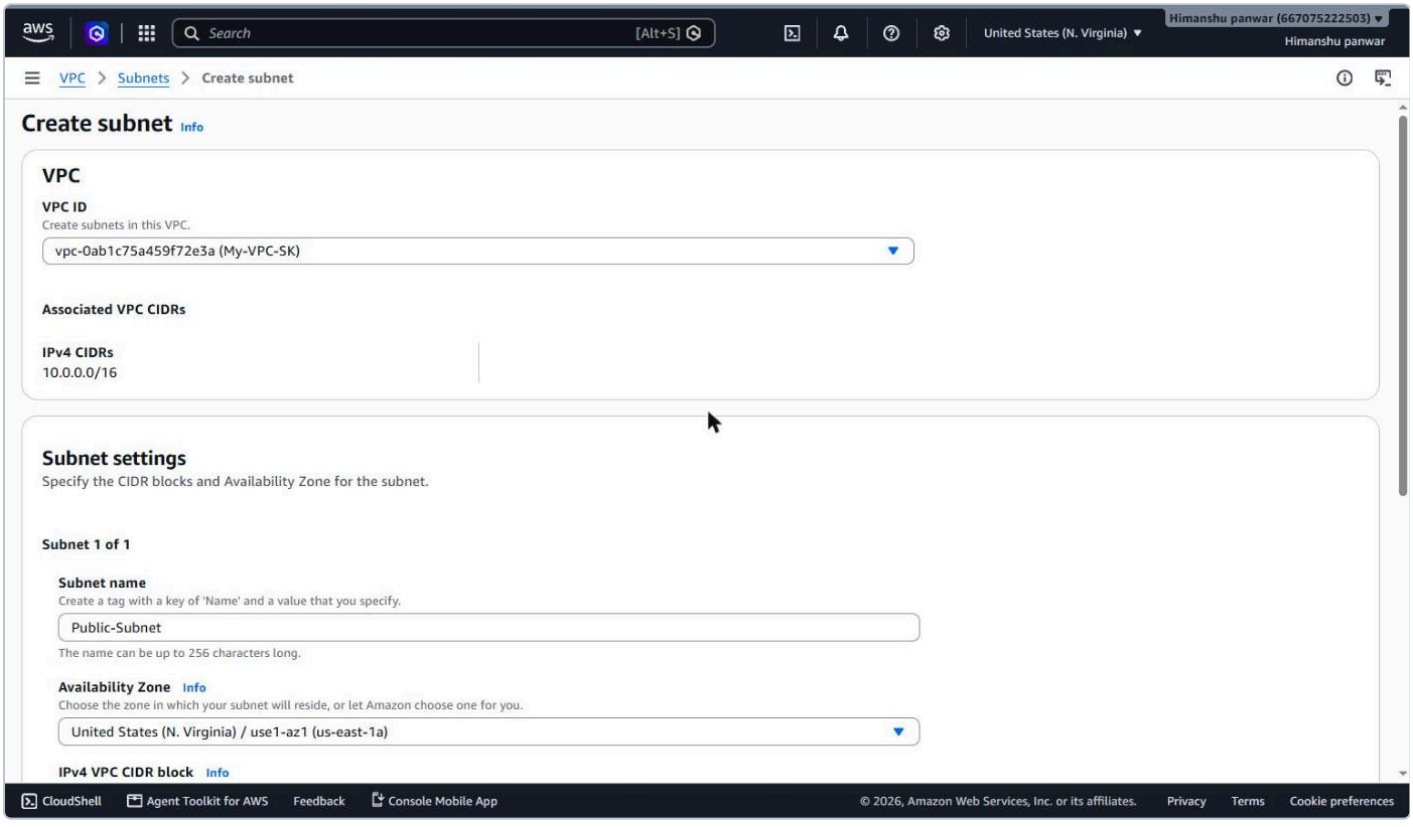


Figure 3.1 — Select My-VPC-SK as the parent VPC.

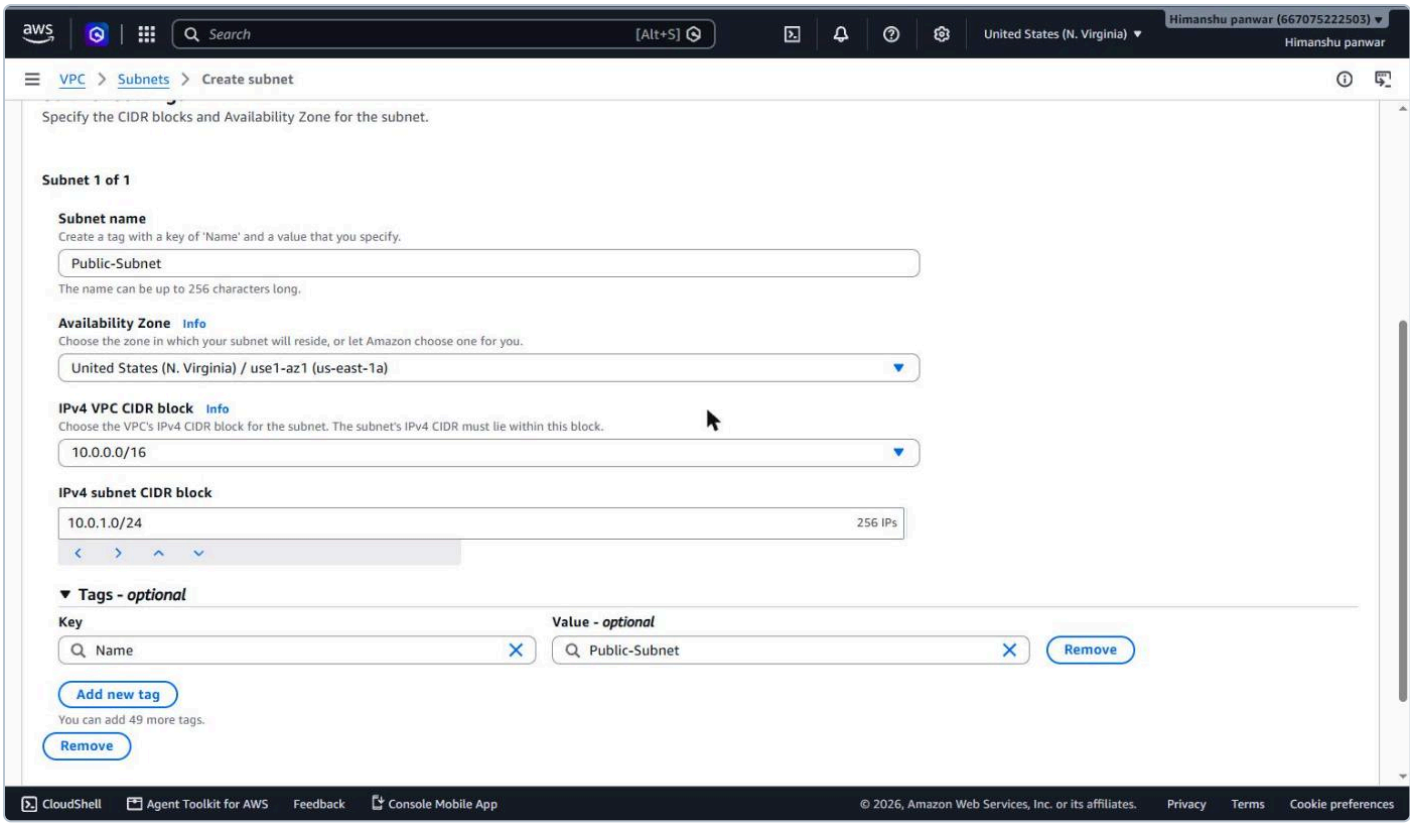


Figure 3.2 — Subnet name, Availability Zone us-east-1a, and CIDR 10.0.1.0/24.

Click path: Click **Create subnet** at the bottom.

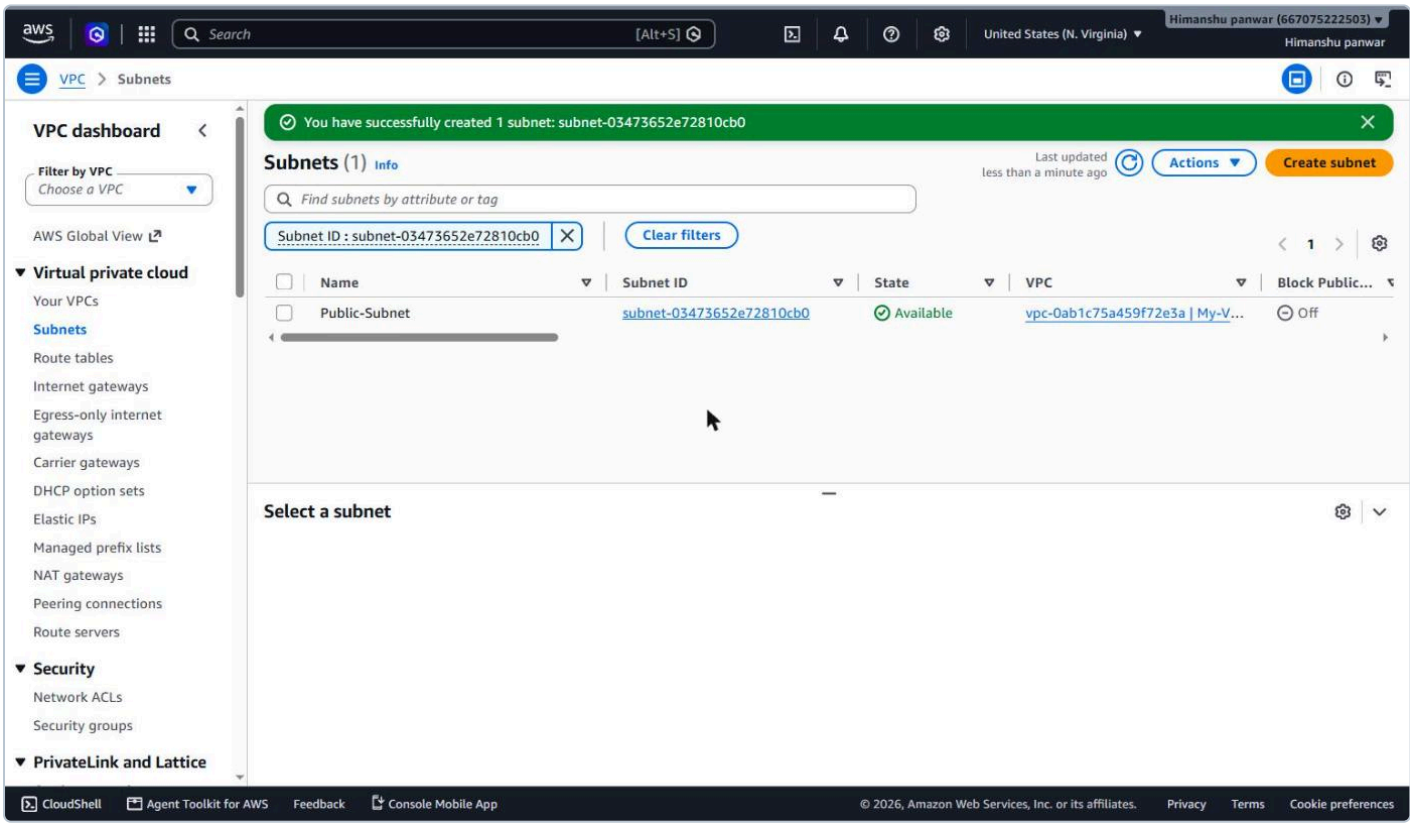


Figure 3.3 — Public-Subnet created successfully with 251 available IPv4 addresses.

Why only 251 usable IPs out of 256?

AWS reserves five addresses in every subnet: the network address (`10.0.1.0`), the VPC router (`10.0.1.1`), the DNS server (`10.0.1.2`), one address reserved for future use (`10.0.1.3`), and the broadcast address (`10.0.1.255`).

4. Step 3 — Create the Private Subnet

The private subnet will hold servers that must never be directly reachable from the internet — databases, application backends, internal APIs.

4.1 Create the Subnet

Click path: Still on the **Subnets** page → click **Create subnet** again.

Field	What to select / type
VPC ID	Select **My-VPC-SK**
Subnet name	Type `Private-Subnet`
Availability Zone	Select **us-east-1a** (same AZ keeps the lab simple)
IPv4 subnet CIDR block	Type `10.0.2.0/24`

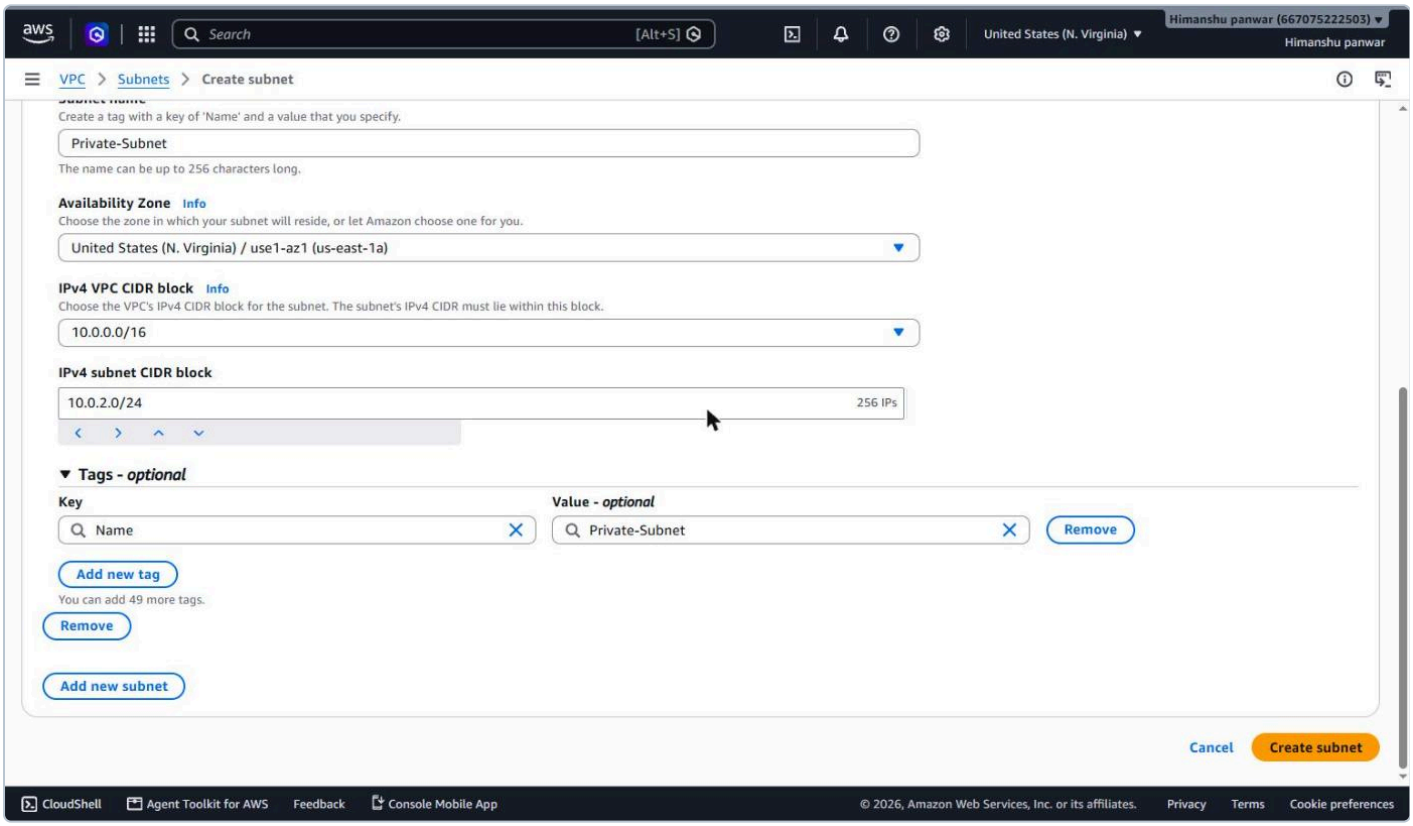


Figure 4.1 — Private-Subnet with CIDR 10.0.2.0/24 in us-east-1a.

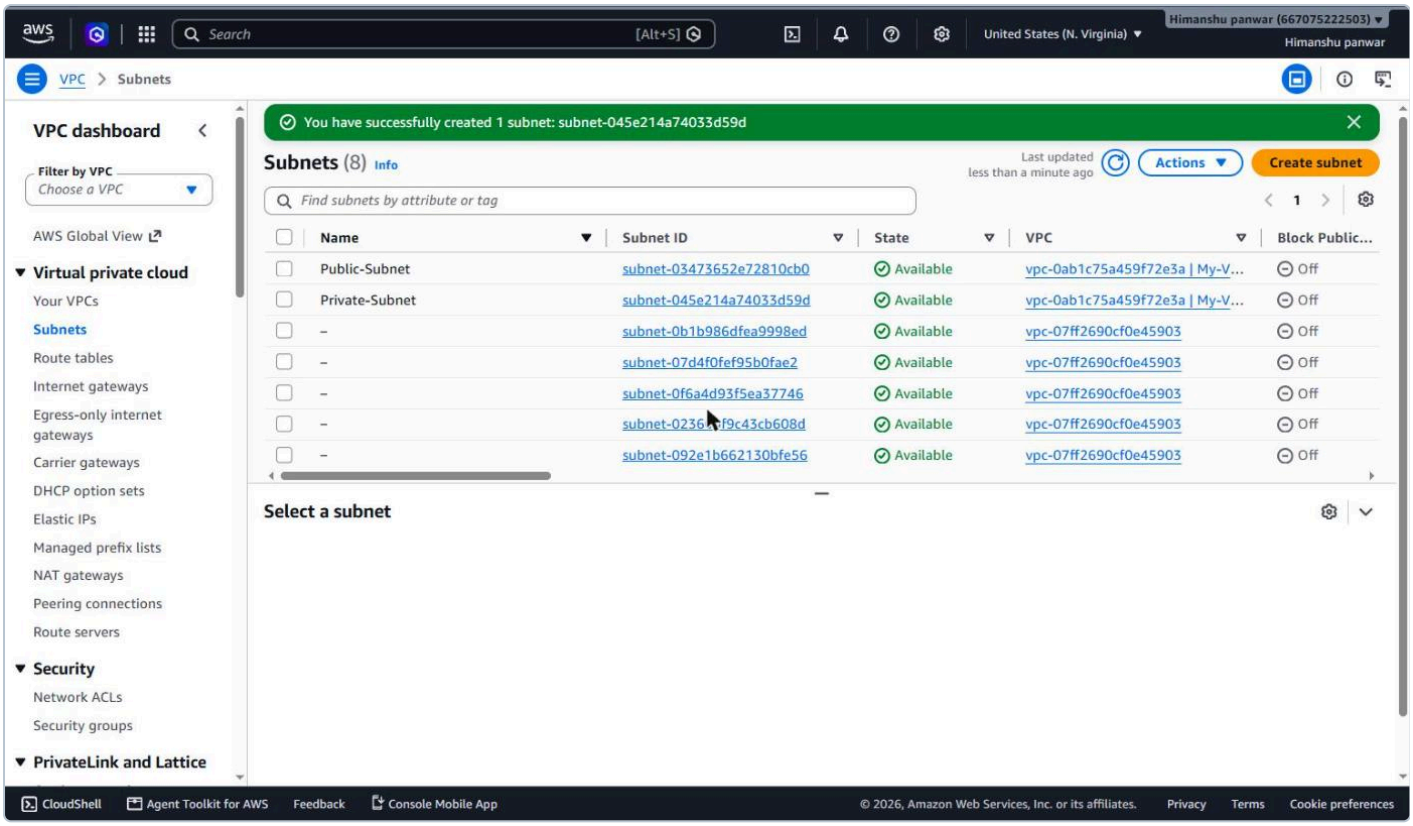


Figure 4.2 — Both subnets now exist inside My-VPC-SK.

Checkpoint

The Subnets list should show **Public-Subnet (10.0.1.0/24)** and **Private-Subnet (10.0.2.0/24)**, both belonging to My-VPC-SK. At this moment *both are private* — neither has a route to the internet yet.

5. Step 4 — Create and Attach the Internet Gateway

An **Internet Gateway (IGW)** is the door between your VPC and the public internet. It is horizontally scaled, redundant, and highly available — AWS manages it for you. One VPC can have only one IGW attached.

5.1 Create the Internet Gateway

Click path: Left sidebar → **Internet gateways** → click **Create internet gateway**.

Field	What to type
Name tag	Type `My-IGW`

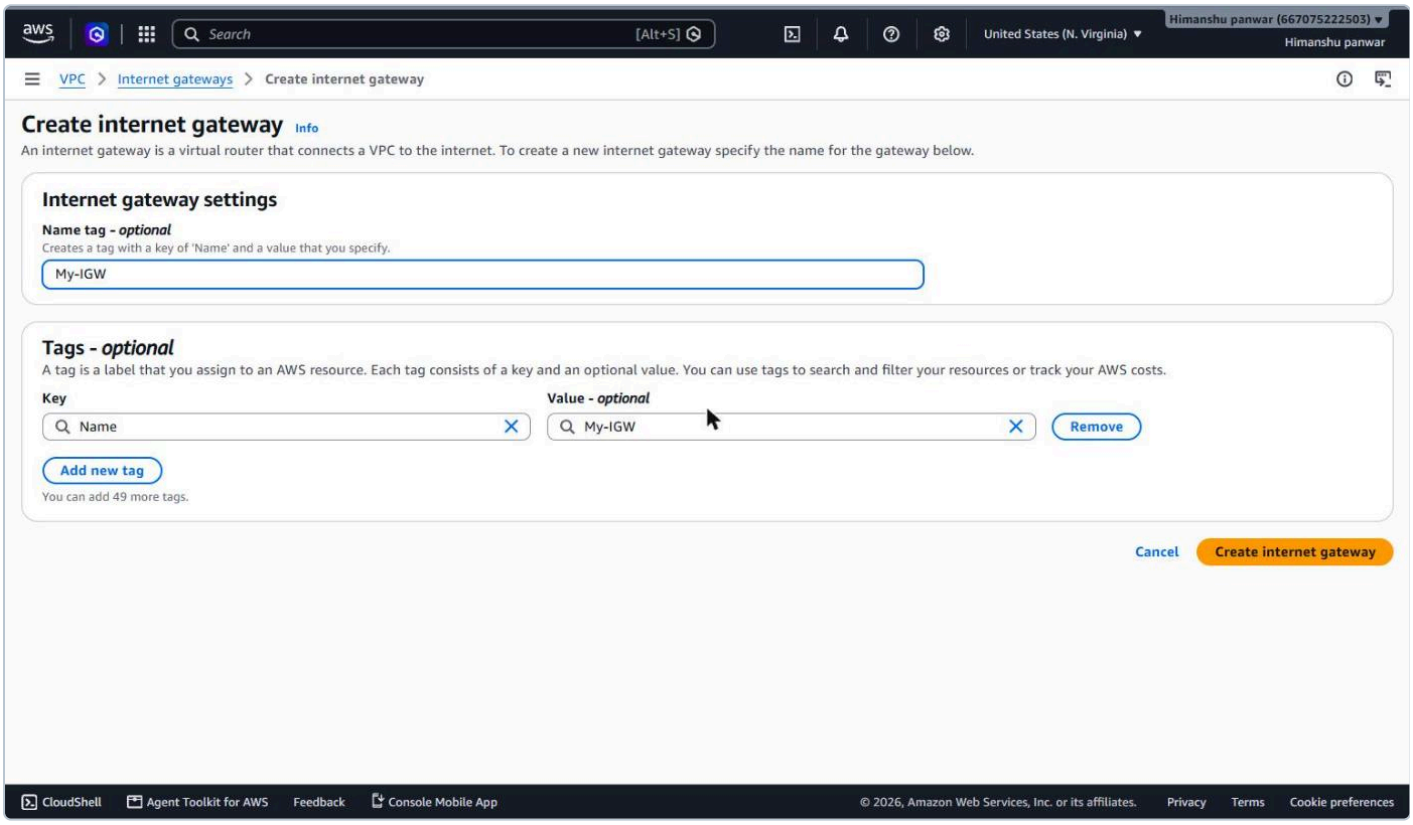


Figure 5.1 — Only a name tag is required to create an IGW.

Click path: Click **Create internet gateway**.

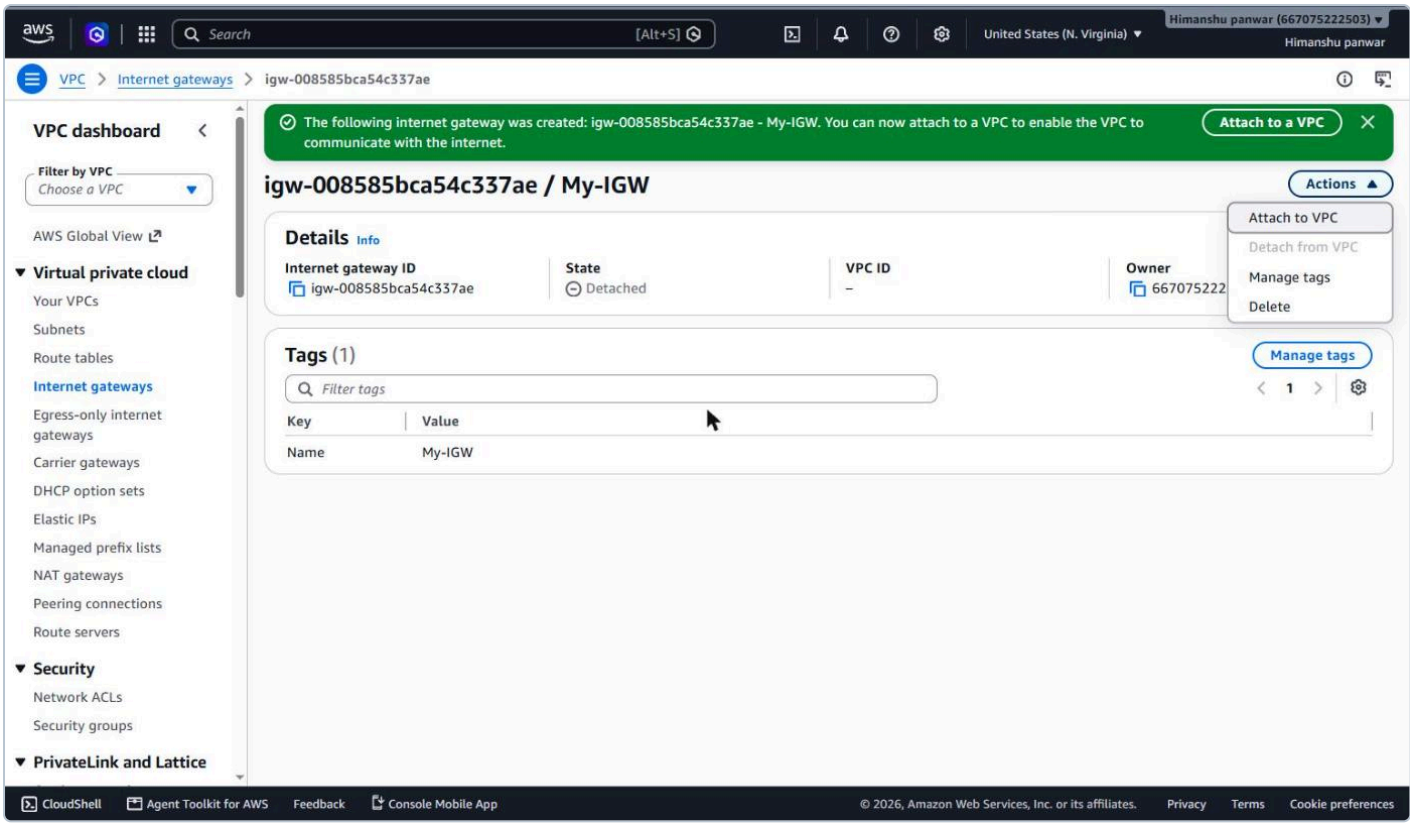


*Figure 5.2 — IGW created, but State = **Detached**. It is not connected to any VPC yet.*

5.2 Attach the IGW to Your VPC

A detached IGW does nothing. You must attach it to My-VPC-SK.

> **Click path:** On the IGW detail page click the **Actions** dropdown (top-right) → select **Attach to VPC**.

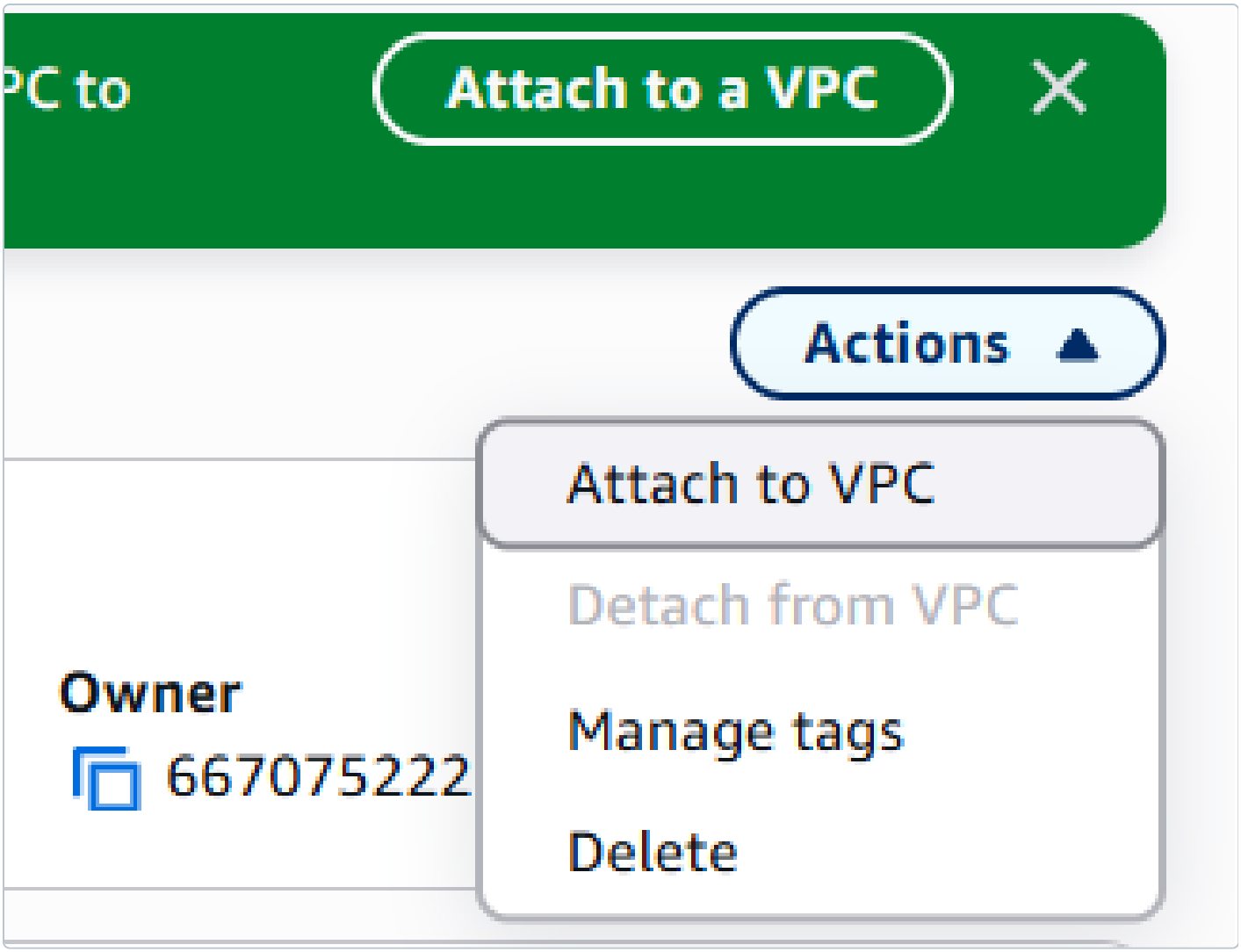


Figure 5.3 — The Actions menu with "Attach to VPC" highlighted.

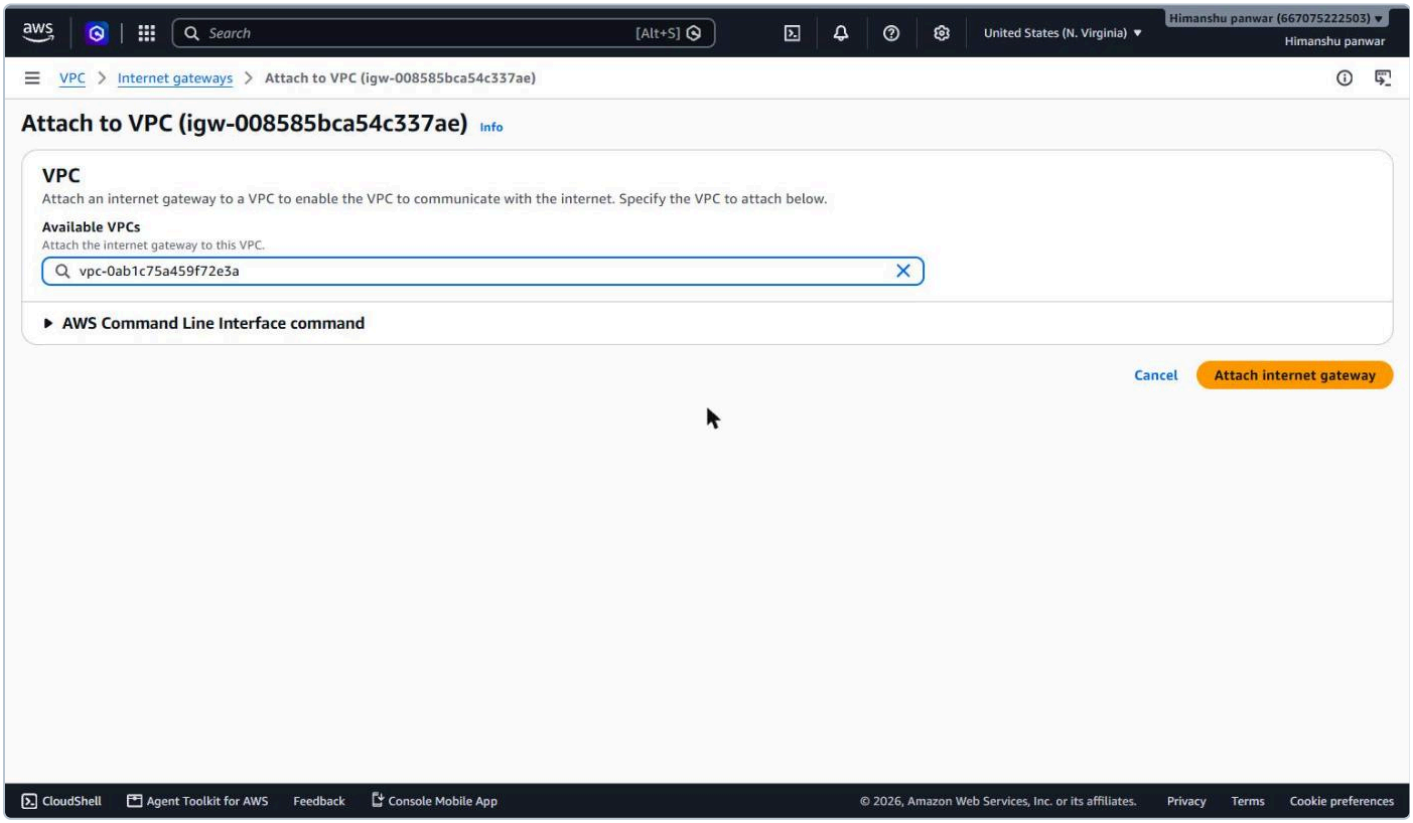


*Figure 5.4 — Select My-VPC-SK from the Available VPCs dropdown, then click **Attach internet gateway**.*

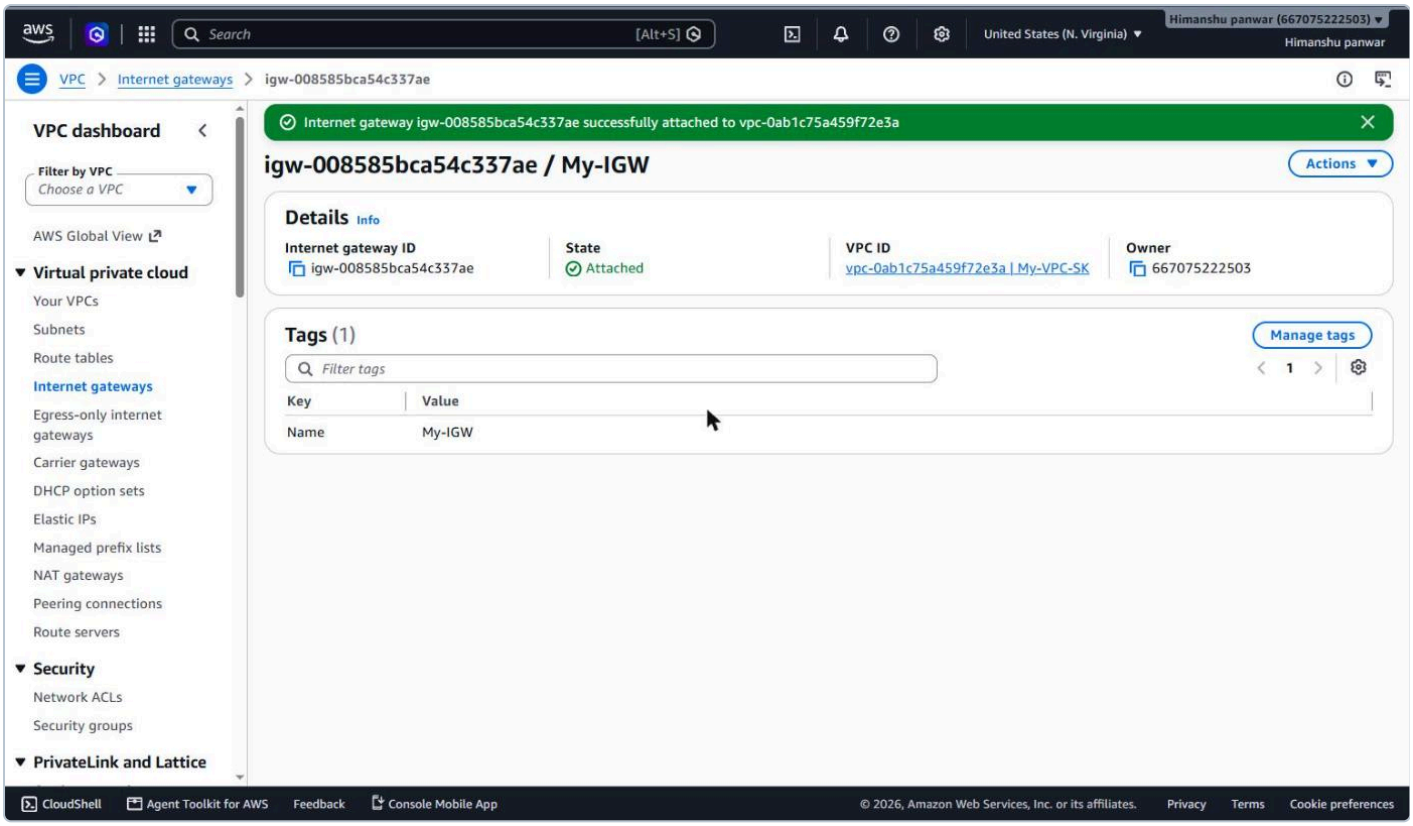


*Figure 5.5 — State is now **Attached** and the VPC ID column shows My-VPC-SK.*

Important

Attaching the IGW does **not** by itself give any subnet internet access. Traffic still needs a *route* pointing at the IGW. That is Step 5.

6. Step 5 — Create the Public Route Table

A **route table** is a set of rules that decides where network traffic goes. Every subnet is associated with exactly one route table. This is the step that actually turns Public-Subnet into a *public* subnet.

6.1 Create the Route Table

Click path: Left sidebar → **Route tables** → click **Create route table**.

Field	What to select / type
Name	Type `Public-RT`
VPC	Select **My-VPC-SK**

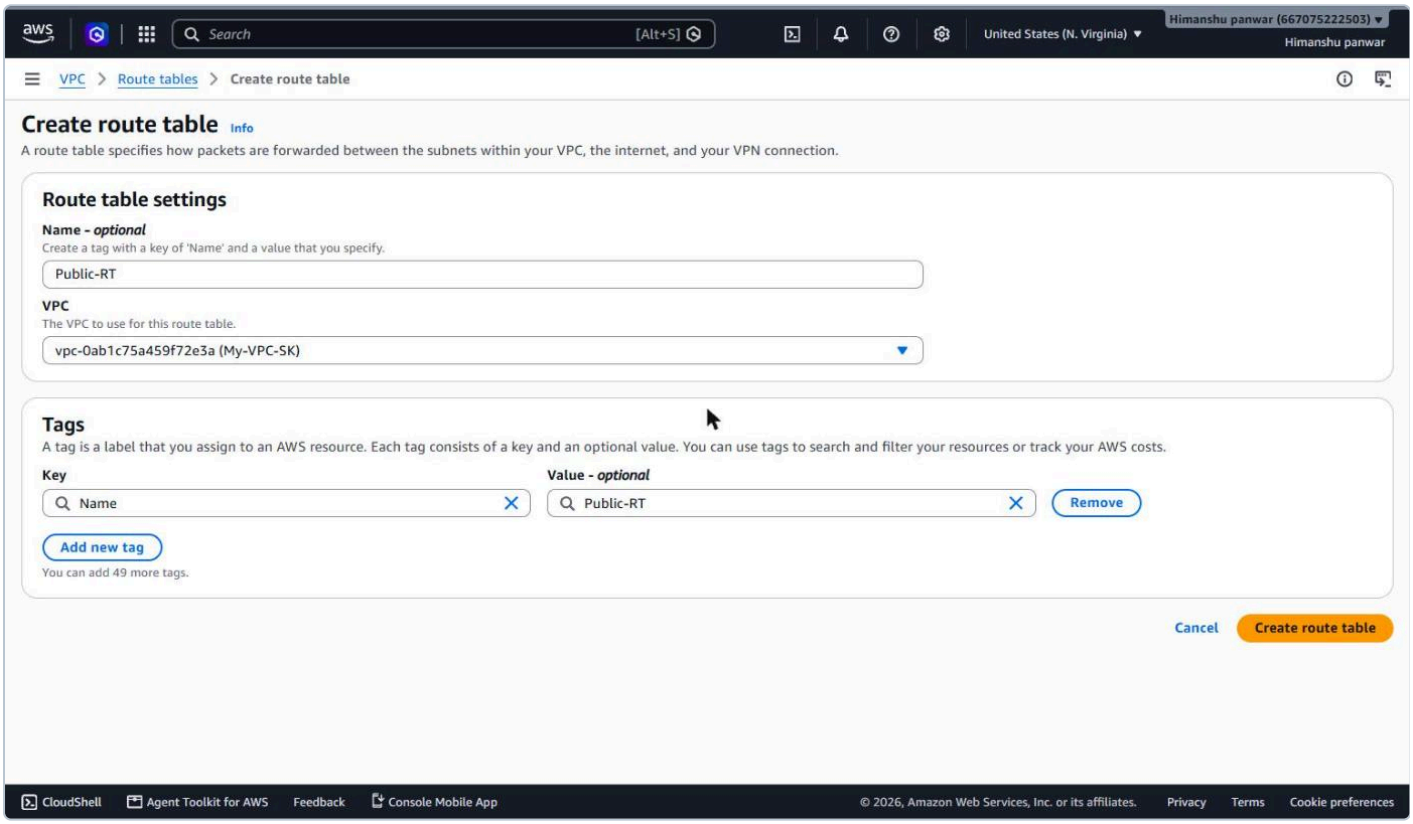


Figure 6.1 — Name the route table Public-RT and attach it to My-VPC-SK.

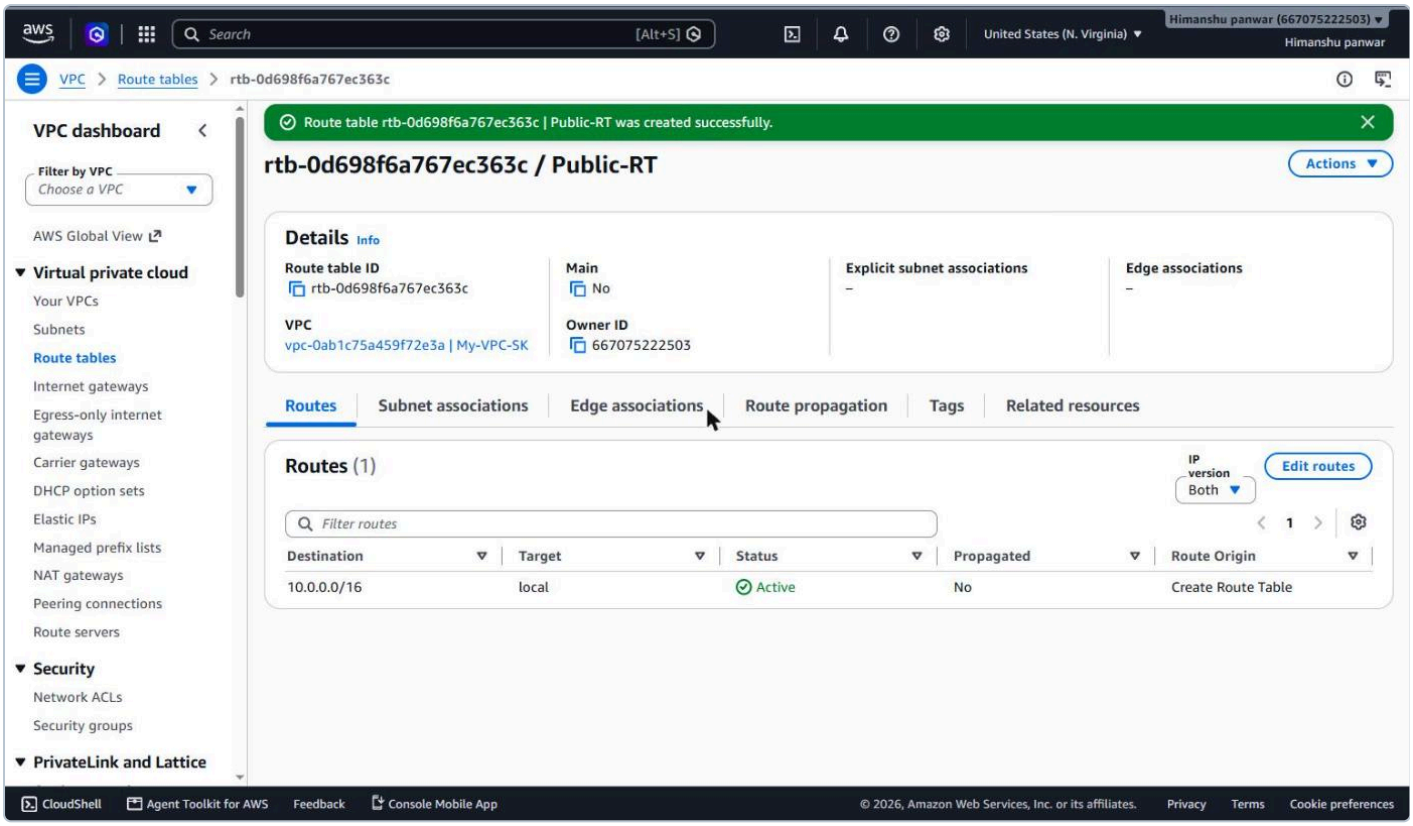


Figure 6.2 — Public-RT created. It starts with only the `10.0.0.0/16 → local` route.

> **What is the "local" route?**
> Every route table automatically contains `10.0.0.0/16 → local`. This is what lets all resources inside your VPC talk to each other. It cannot be deleted or modified.

6.2 Add the Internet Route

Click path: With Public-RT selected, click the **Routes** tab → click **Edit routes** → click **Add route**.

Field	What to select / type
Destination	Type `0.0.0.0/0` (means "anywhere on the internet")
Target	Click the dropdown → select **Internet Gateway** → select **My-IGW**

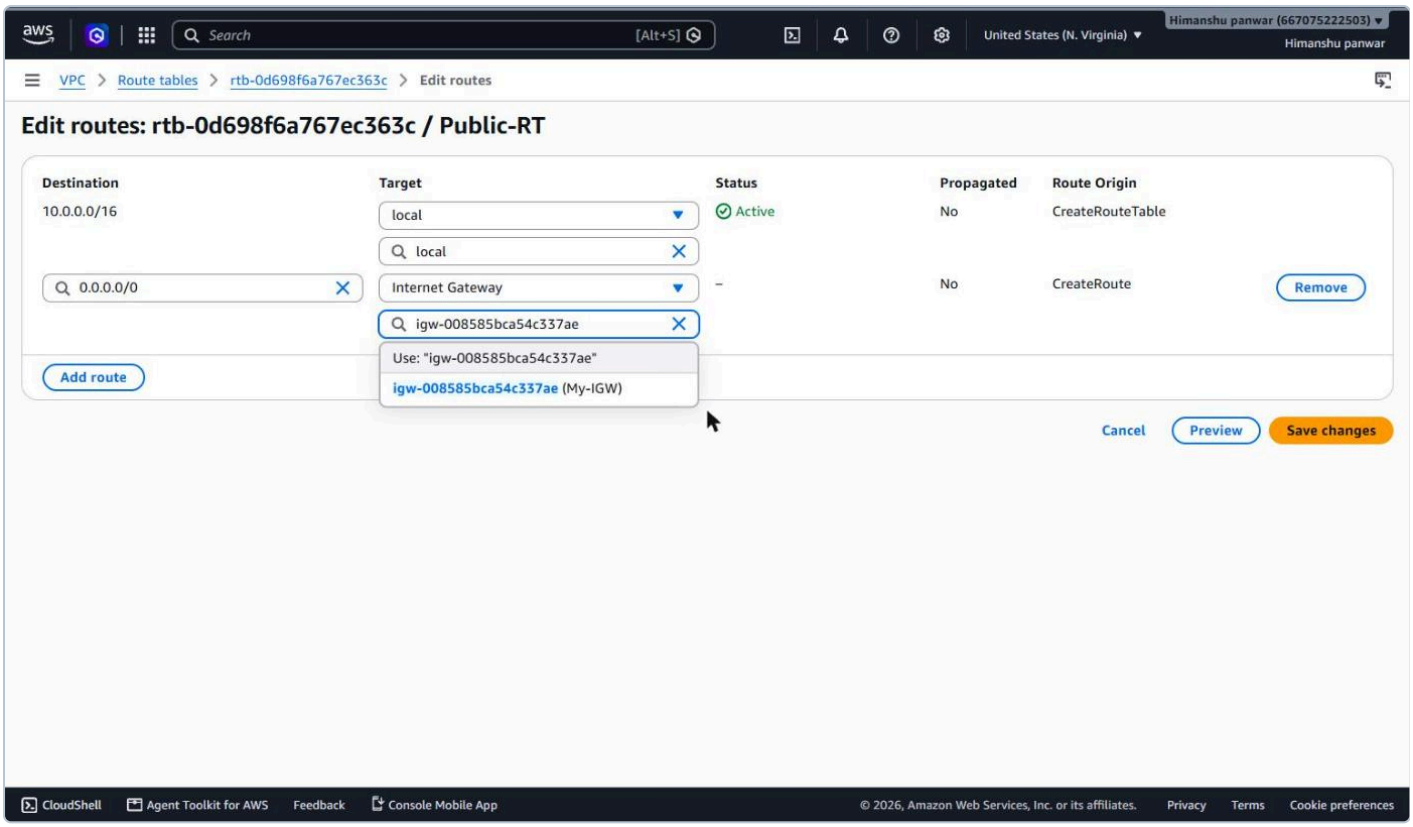


Figure 6.3 — Adding the default route `0.0.0.0/0 → My-IGW`.

Click path: Click **Save changes**.

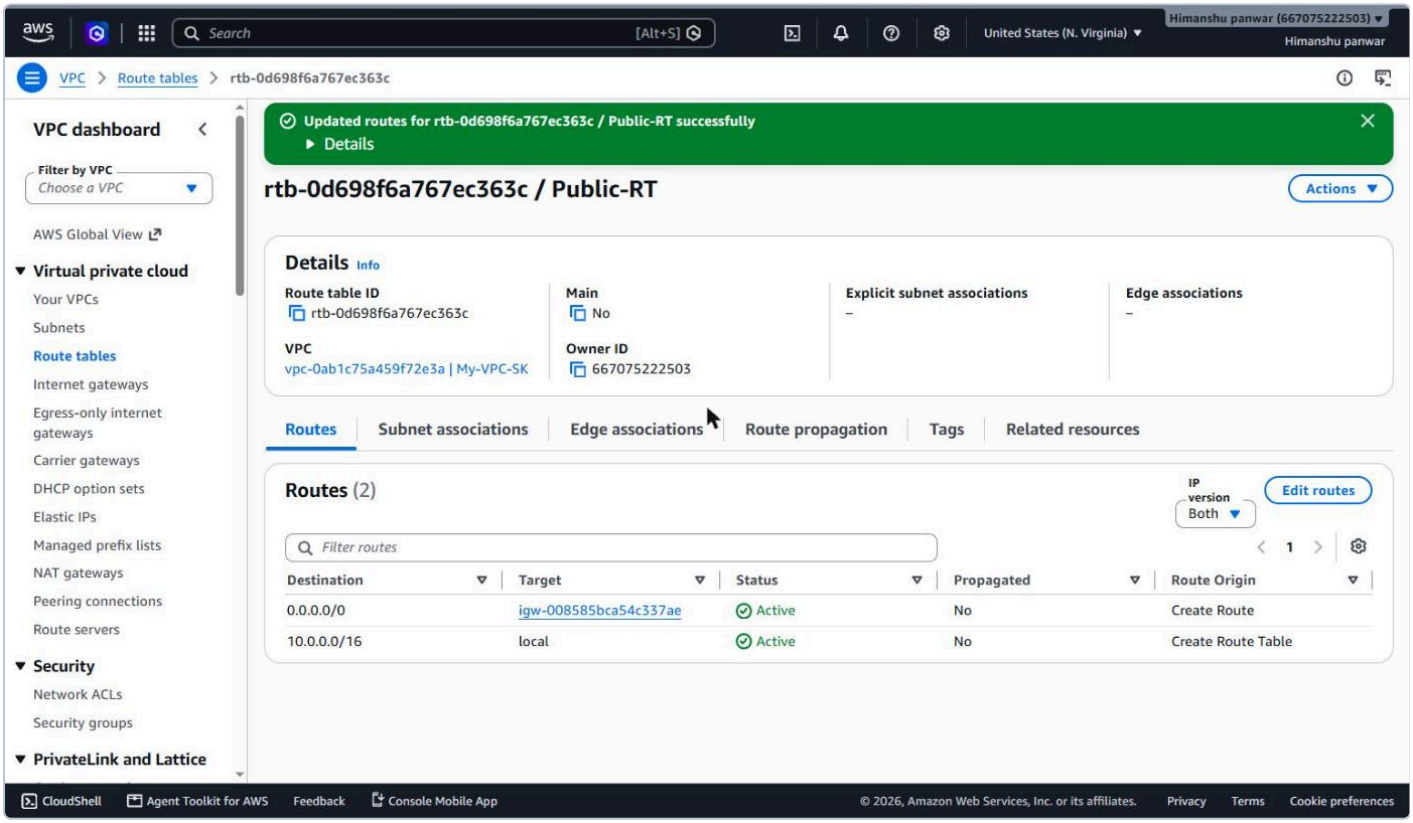


*Figure 6.4 — Both routes are now **Active**: local for internal traffic, IGW for internet traffic.*

6.3 Associate Public-Subnet with Public-RT

The route table exists, but no subnet is using it yet. Associate it with Public-Subnet.

Click path: Click the **Subnet associations** tab → click **Edit subnet associations** → tick the checkbox next to **Public-Subnet** → click **Save associations**.

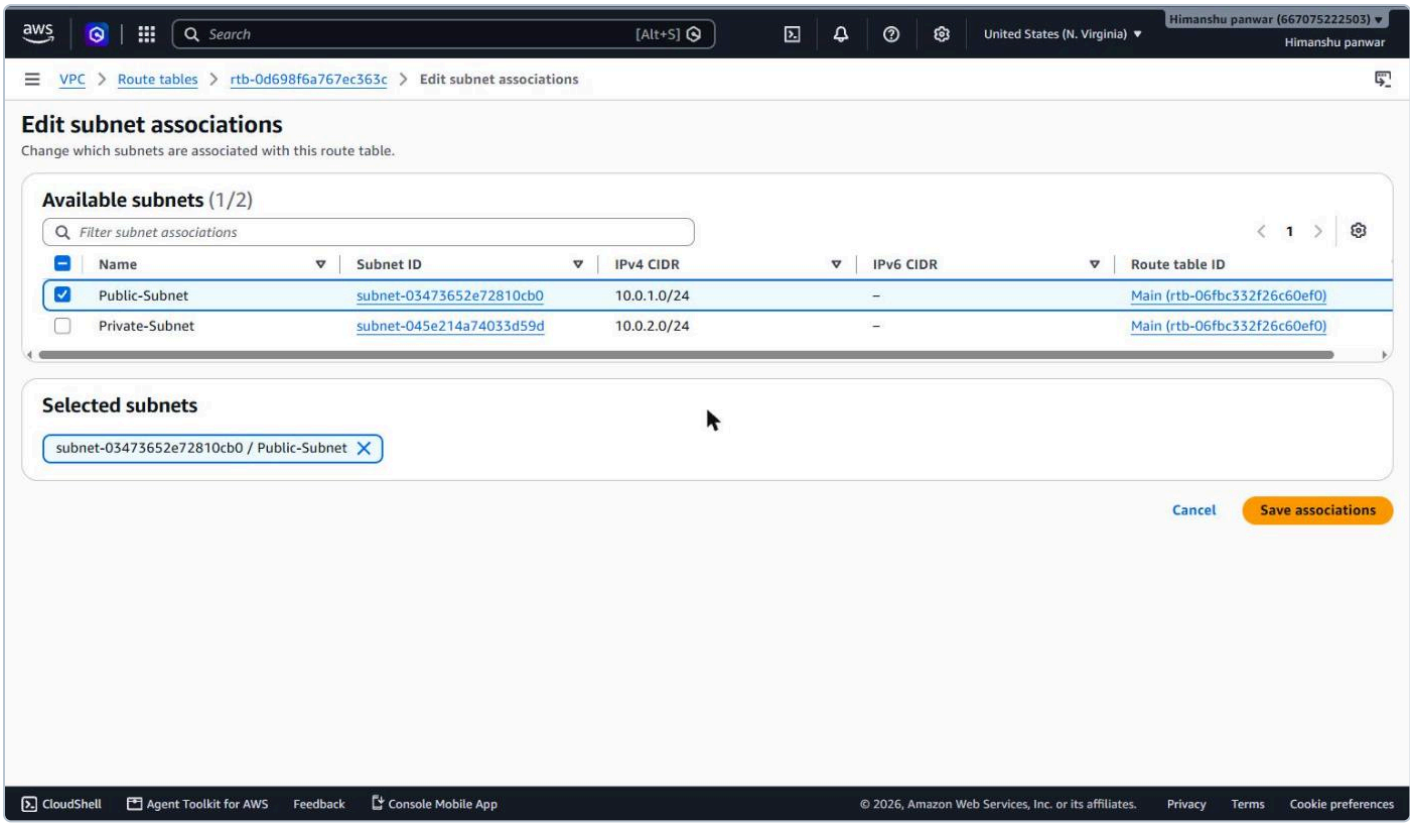


*Figure 6.5 — Tick only Public-Subnet. Do **not** tick Private-Subnet here.*

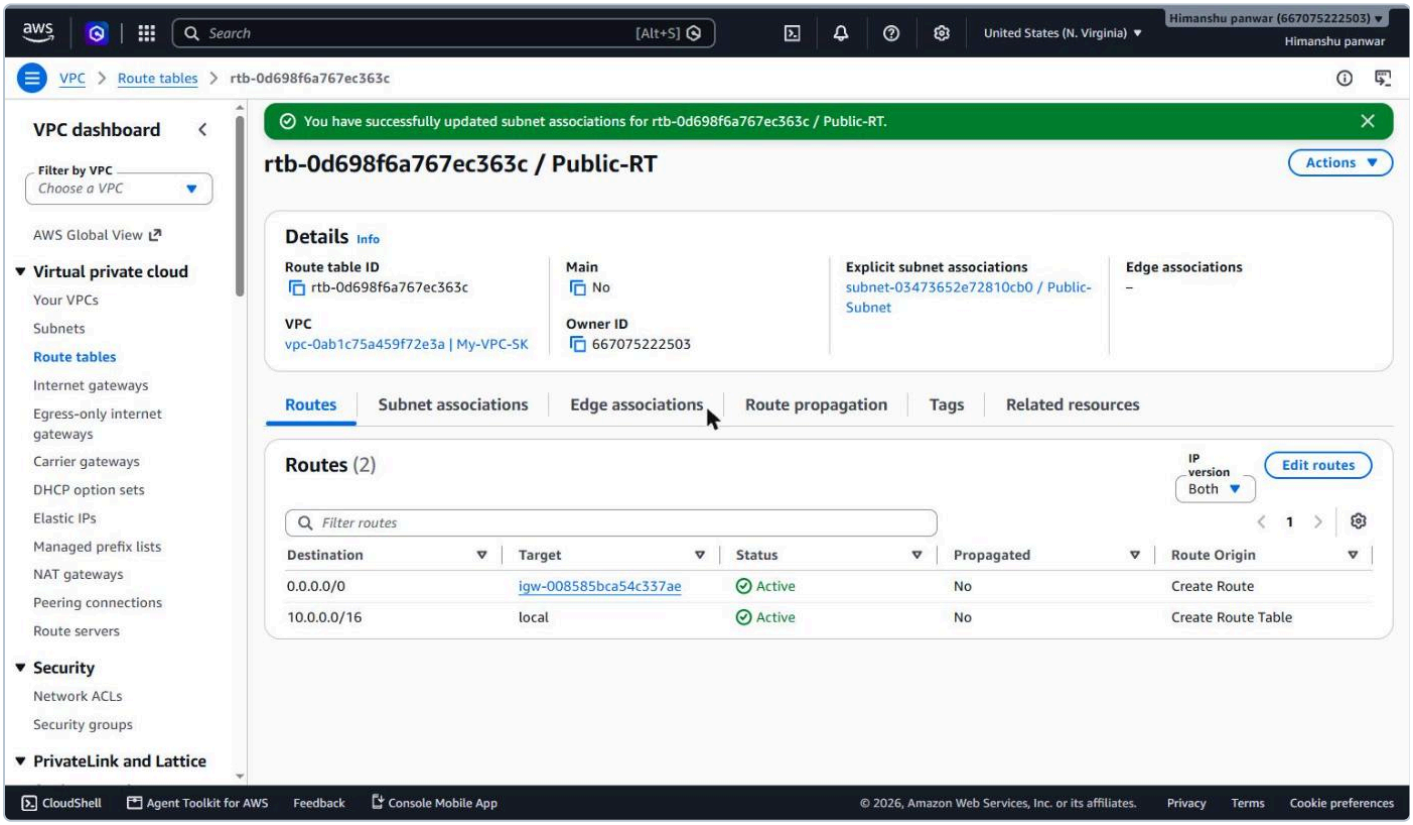


Figure 6.6 — Public-RT is complete: 2 routes and 1 explicit subnet association.

Checkpoint

Public-Subnet is now genuinely public. Any instance launched there with a public IP can reach the internet and be reached from it.

7. Step 6 — Create the Private Route Table

We create a dedicated route table for the private subnet so we can control its outbound path separately. For now it will have **no internet route at all** — that is exactly what makes it private.

7.1 Create the Route Table

Click path: Route tables → Create route table.

Field	What to select / type
Name	Type `Private-RT`
VPC	Select **My-VPC-SK**

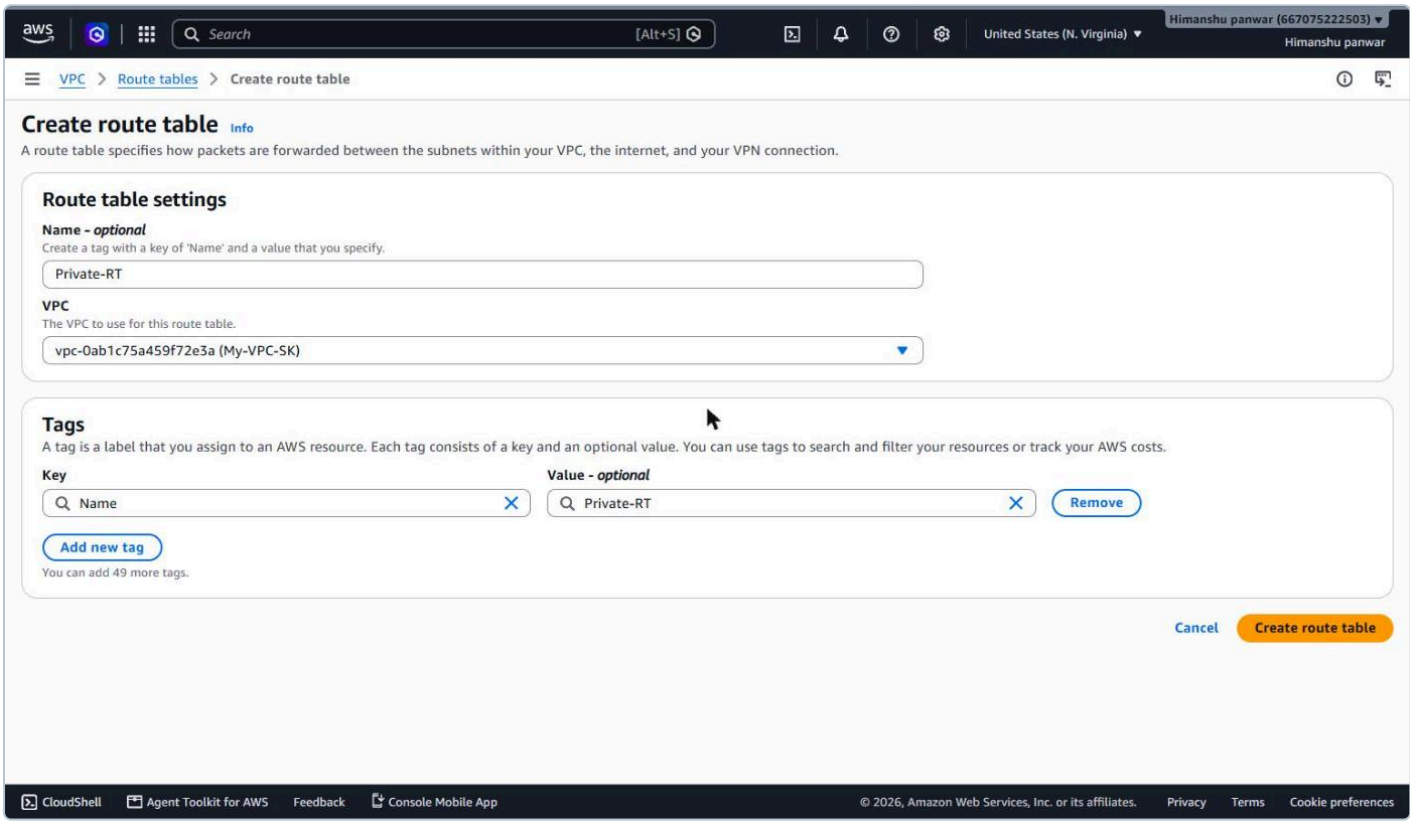


Figure 7.1 — Creating Private-RT inside My-VPC-SK.

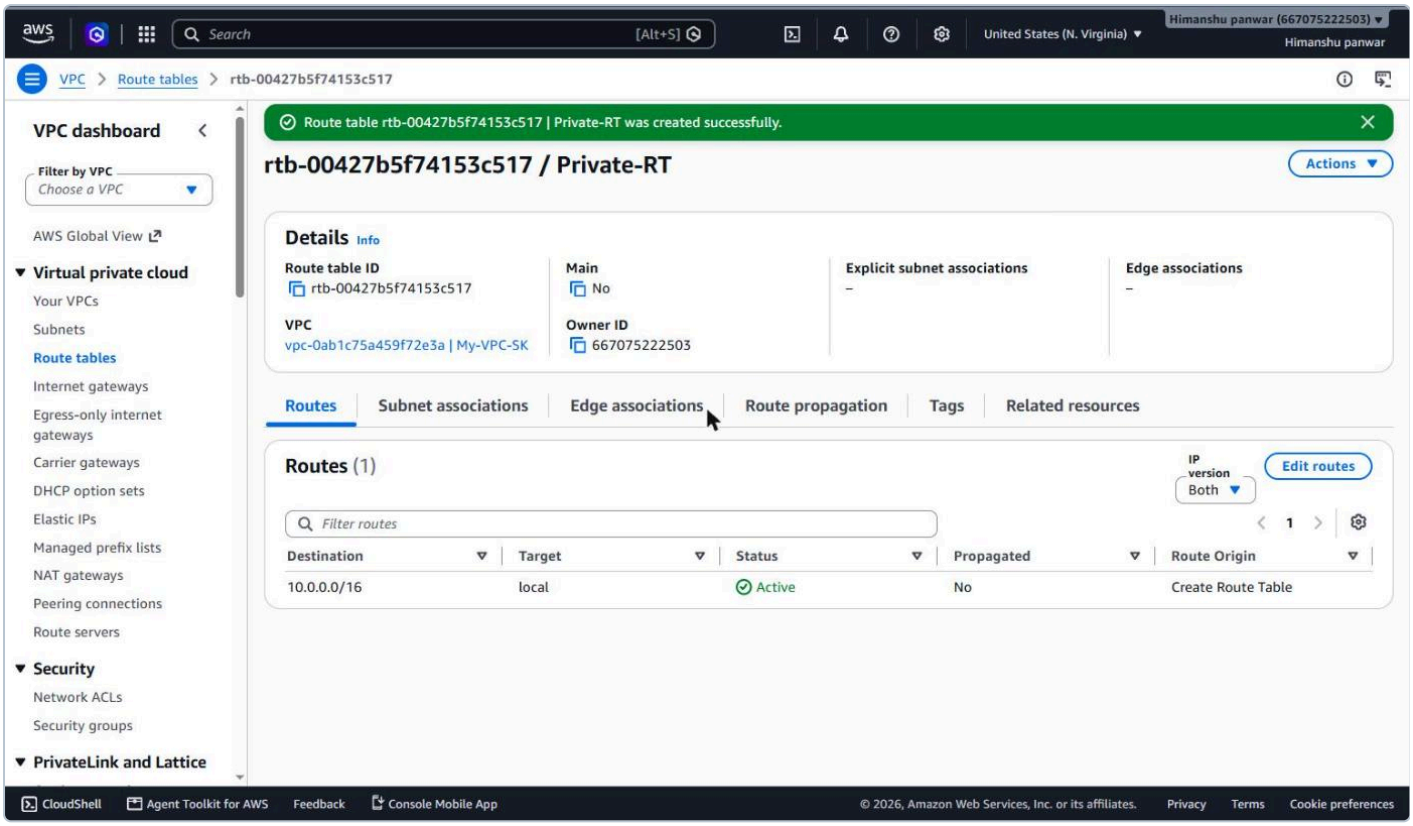


Figure 7.2 — Private-RT has only the local route. ***Do not add an IGW route here.***

7.2 Associate Private-Subnet with Private-RT

Click path: Subnet associations tab → **Edit subnet associations** → tick **Private-Subnet** → **Save associations**.

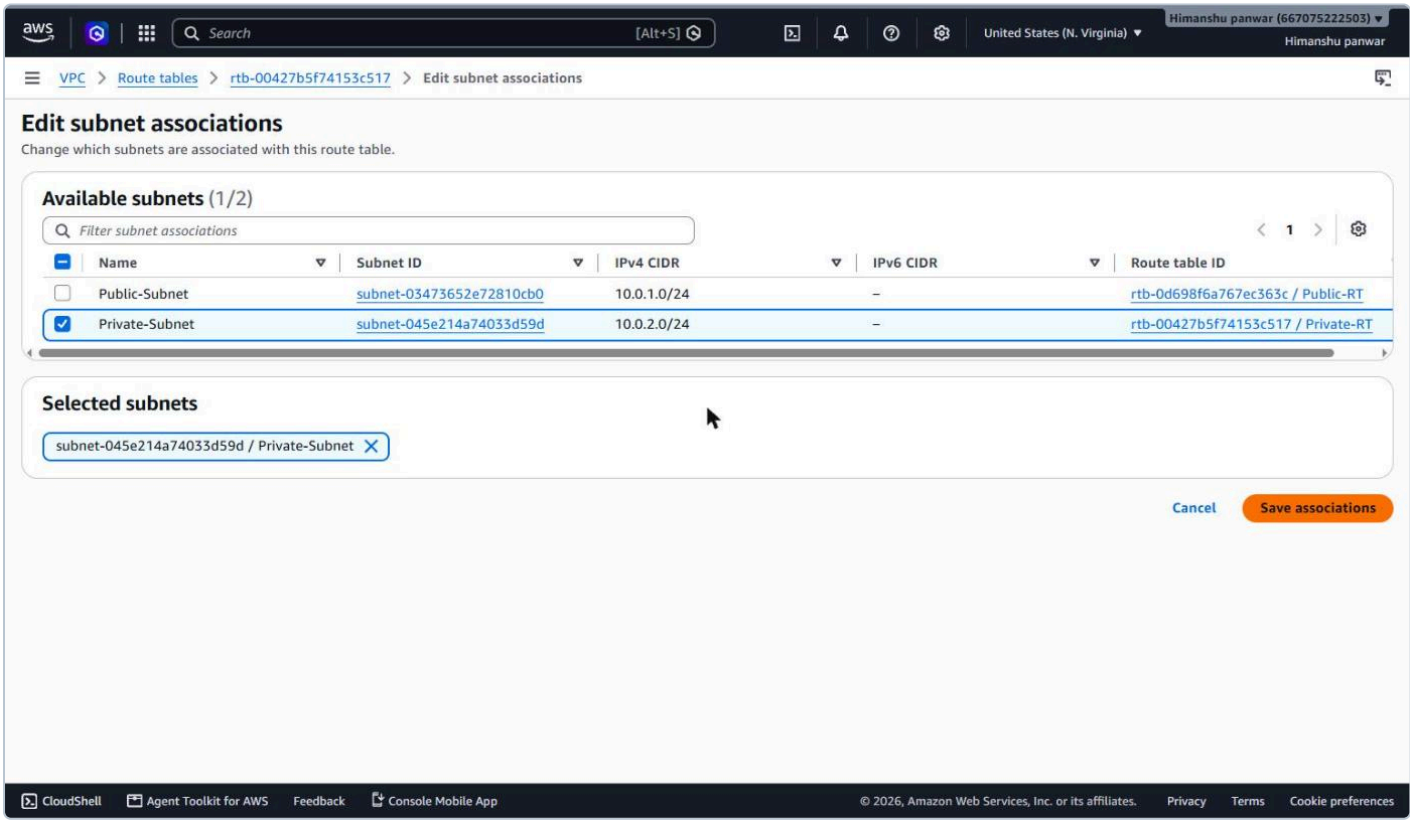


Figure 7.3 — Tick only Private-Subnet.

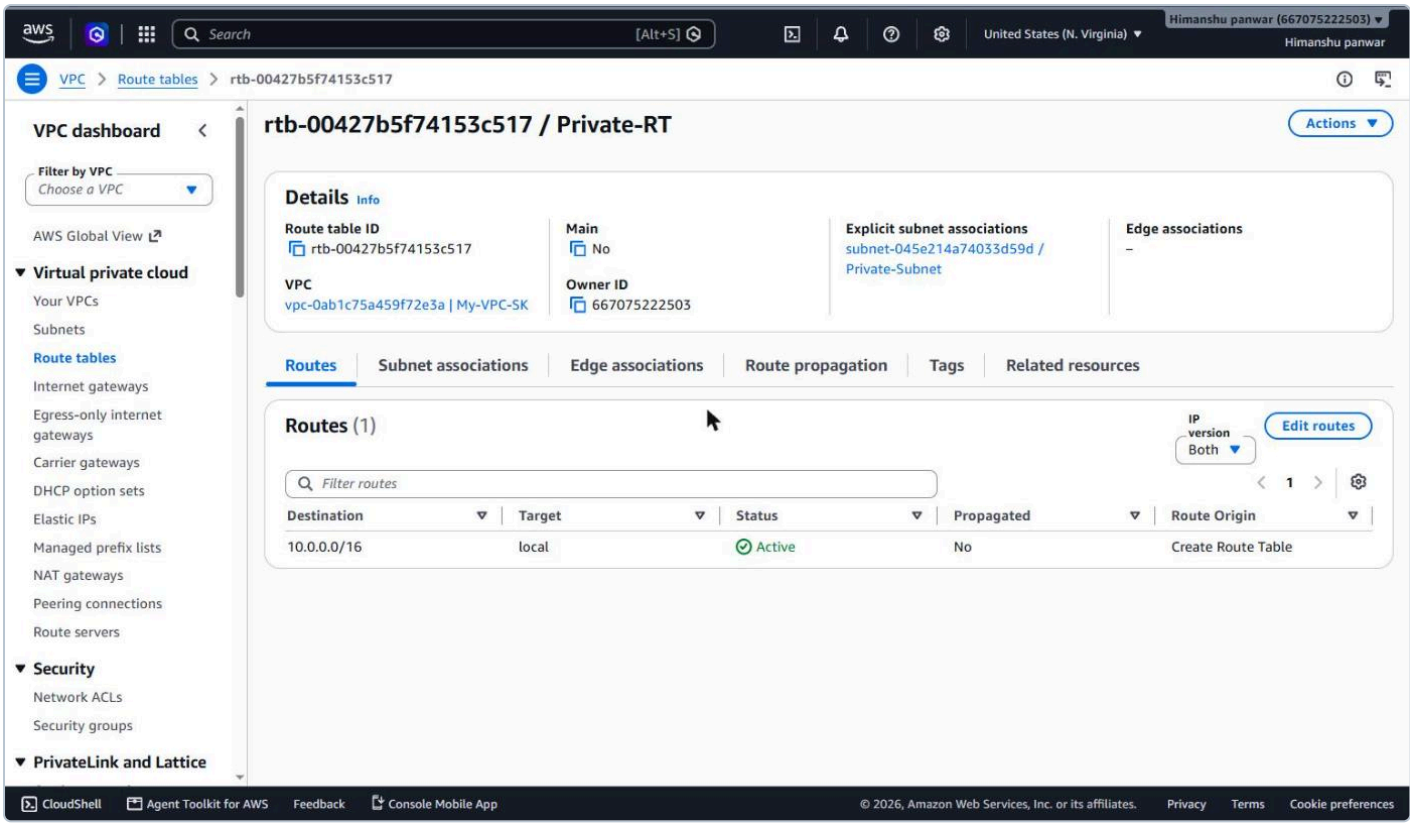


Figure 7.4 — Private-RT: 1 route (local only), 1 subnet association.

7.3 Compare the Two Route Tables

	Public-RT	Private-RT
Routes	`10.0.0.0/16 → local` `0.0.0.0/0 → igw-xxxx`	`10.0.0.0/16 → local`
Associated subnet	Public-Subnet	Private-Subnet
Internet access	YES both ways	NO neither way

8. Step 7 — Create the Security Group

A **Security Group (SG)** is a stateful virtual firewall attached to an instance's network interface. "Stateful" means that if you allow traffic in, the reply is automatically allowed out, and vice versa — you do not need a matching rule for return traffic.

8.1 Create the Security Group

Click path: Left sidebar → under **Security** click **Security groups** → click **Create security group**.

Field	What to type / select
Security group name	Type `My-SG`
Description	Type `Allow SSH and HTTP`
VPC	Select **My-VPC-SK** (very important — do not leave the default VPC)

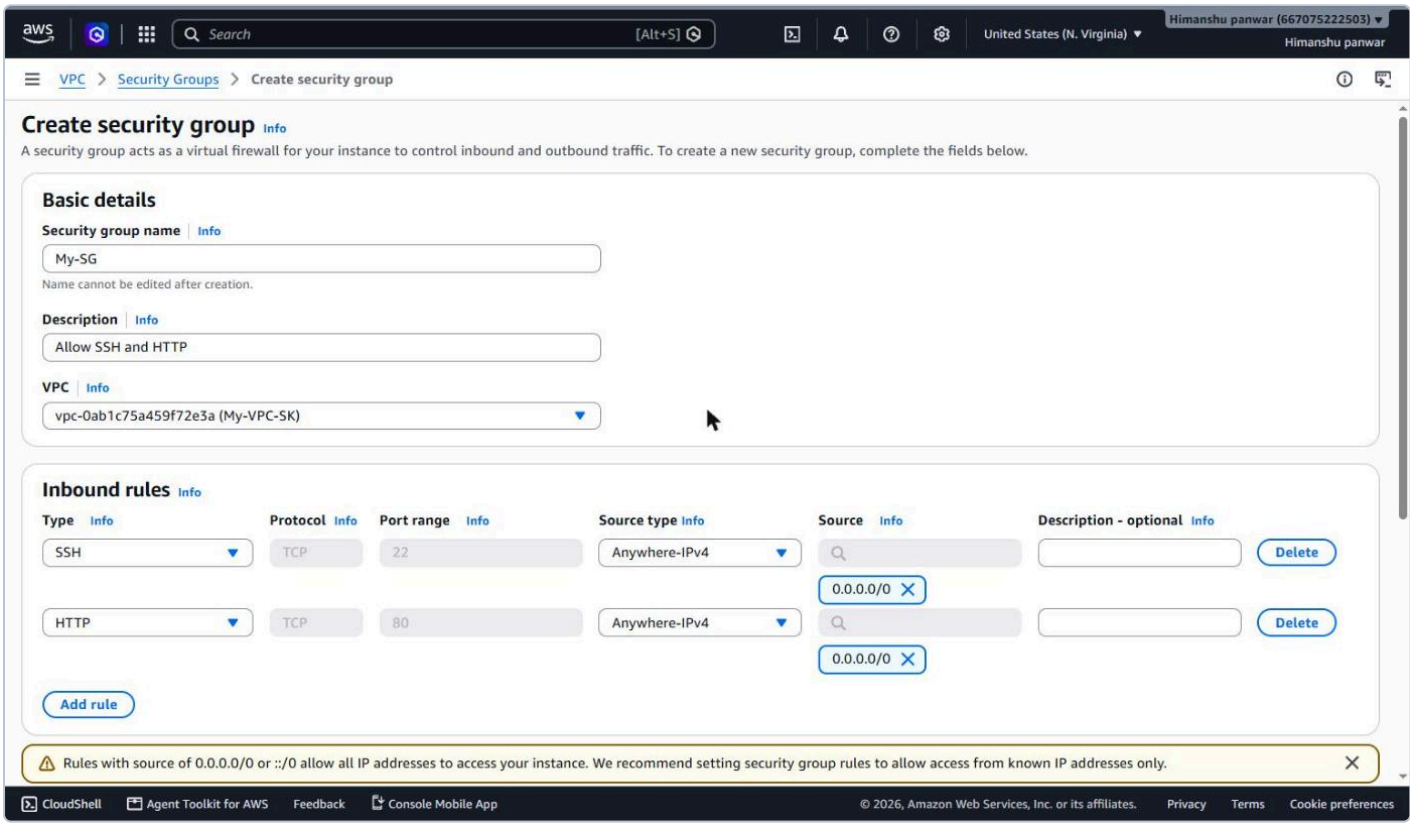


Figure 8.1 — Basic details. Make sure the VPC is My-VPC-SK.

8.2 Add Inbound Rules

Click path: In the **Inbound rules** section click **Add rule** twice and fill in the two rows below.

Rule	Type	Protocol	Port	Source
1	SSH	TCP	22	Anywhere-IPv4 → `0.0.0.0/0`
2	HTTP	TCP	80	Anywhere-IPv4 → `0.0.0.0/0`

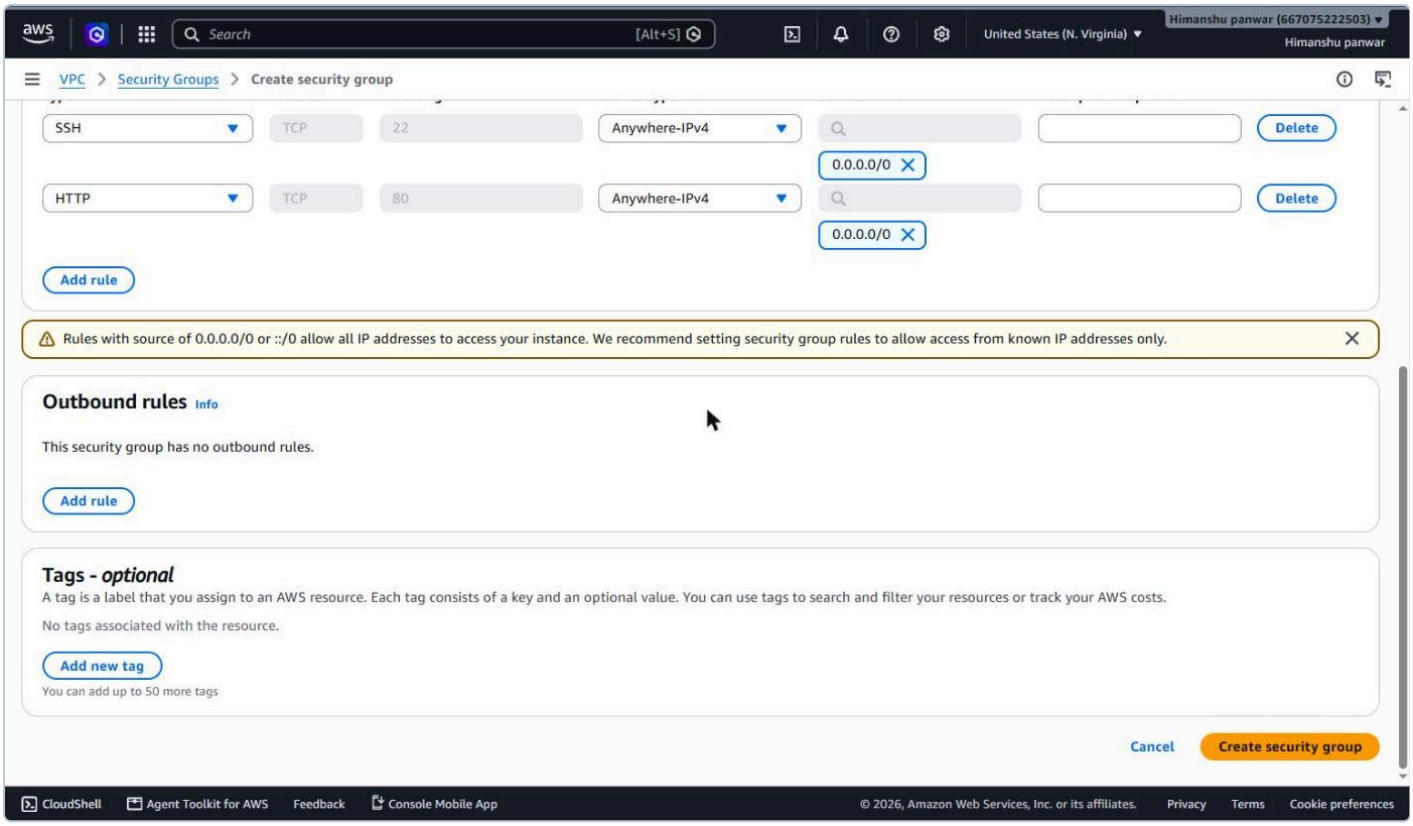


Figure 8.2 — Two inbound rules: SSH (22) and HTTP (80) from 0.0.0.0/0.

8.3 Verify the Outbound Rule

Critical — read this before clicking Create

Scroll down to the **Outbound rules** section and confirm there is **one rule** present: **Type = All traffic, Destination = 0.0.0.0/0**.

If the Outbound rules section is **empty**, click **Add rule** and create it manually (Type: **All traffic**, Destination type: **Anywhere-IPv4**, Destination: `0.0.0.0/0`).

An empty outbound list blocks *all* traffic leaving the instance and will break Step 12 of this lab. This is exactly the mistake that happened during this lab — the full diagnosis is in **Section 17: Troubleshooting**.

Click path: Click **Create security group**.

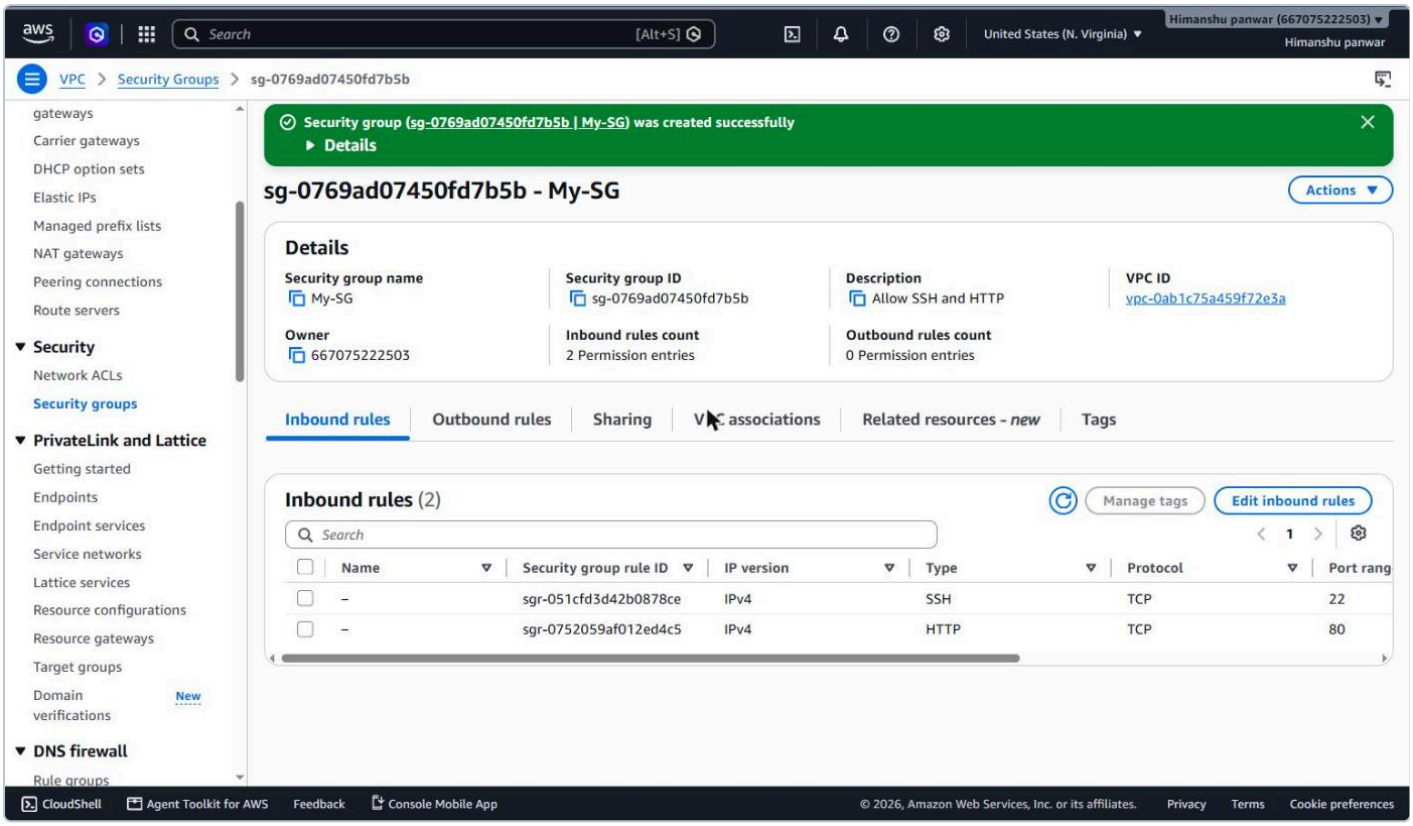


Figure 8.3 — My-SG created. Note the Security group ID (`sg-0769ad07450fd7b5b`) — you will select this when launching instances.

9. Step 8 — Launch the Public EC2 Instance

This instance lives in the public subnet, gets a public IP address, and will act as your **Bastion Host (Jump Server)** — the single controlled entry point into your network.

9.1 Start the Launch Wizard

Click path: In the AWS Console search bar type `EC2` → click **EC2** → left sidebar **Instances** → click **Launch instances**.

9.2 Fill in the Basic Settings

Field	What to select / type
Name	Type `Public-EC2`
Application and OS Image (AMI)	Select **Amazon Linux 2023 AMI** (free-tier eligible)
Instance type	Select **t3.micro** (or t2.micro)
Key pair (login)	Select your existing key pair — this lab uses `test`

9.3 Configure Network Settings — the Critical Part

Click path: In the **Network settings** card click **Edit** (top-right of that card) to reveal all the fields.

Field	What to select
VPC	Select **My-VPC-SK**
Subnet	Select **Public-Subnet** (10.0.1.0/24)
Auto-assign public IP	Set to **Enable** ← must be Enable
Firewall (security groups)	Select **Select existing security group** → tick **My-SG**

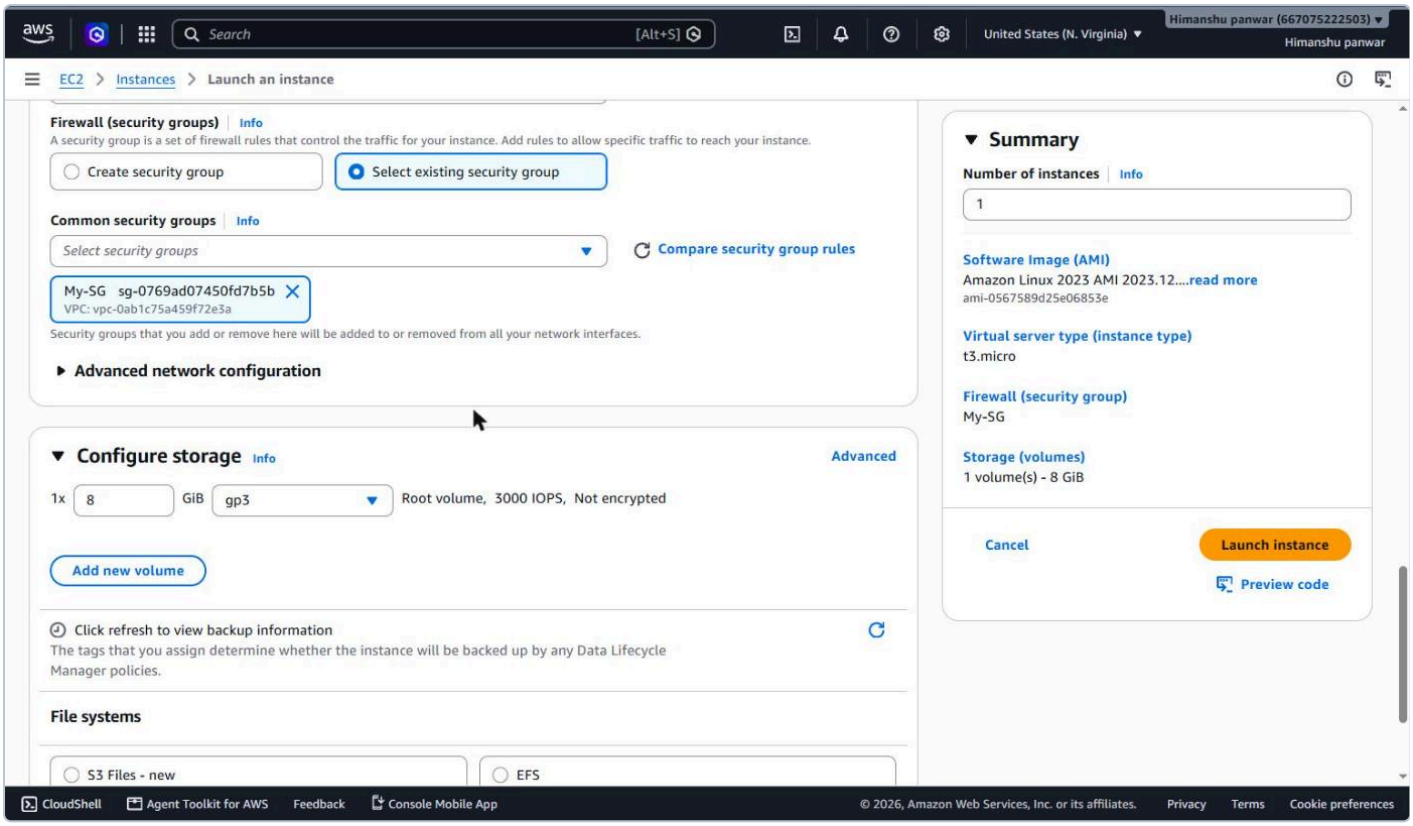


*Figure 9.1 — Public-Subnet selected and Auto-assign public IP set to **Enable**.*

Click path: In the Summary panel on the right, click **Launch instance**.

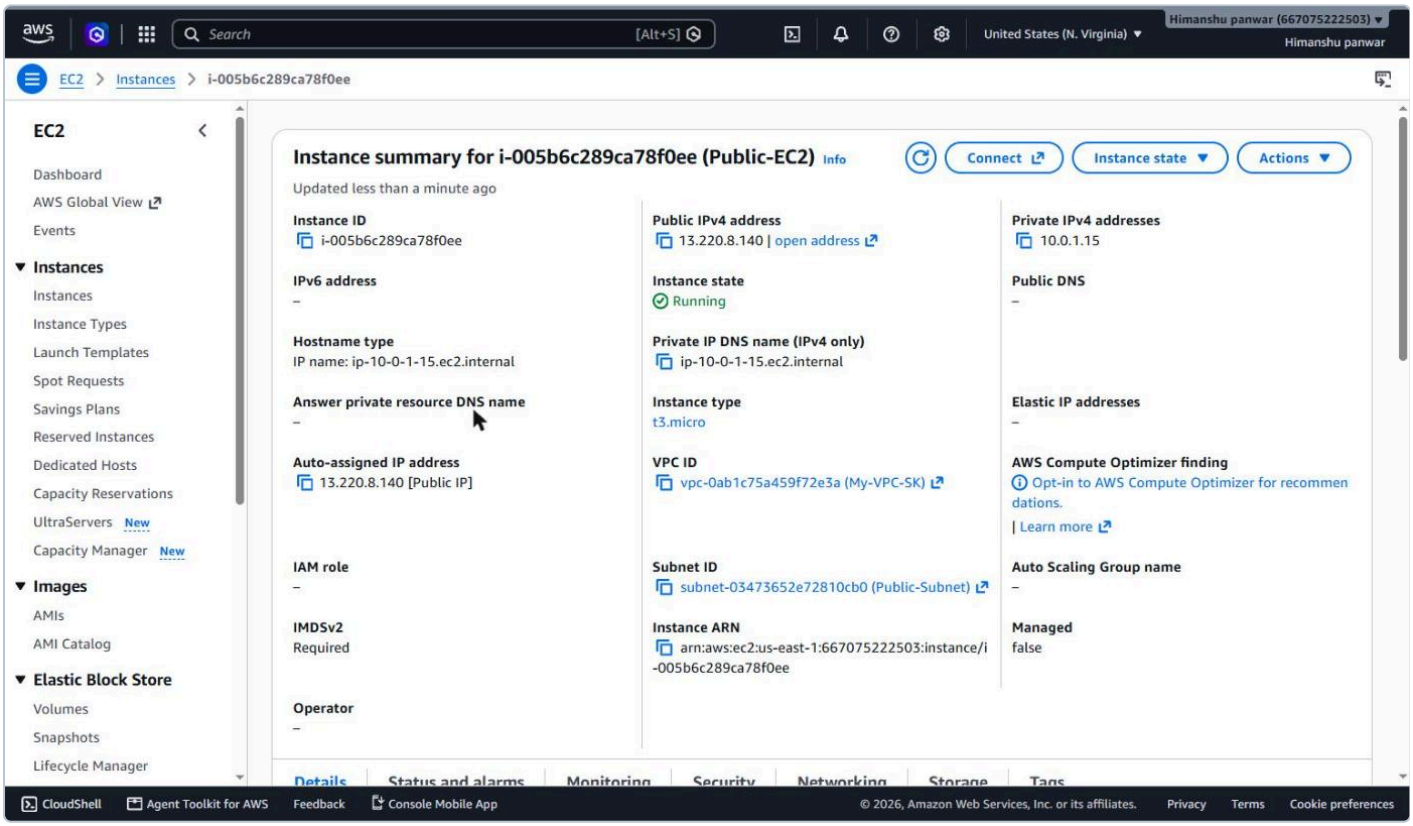


Figure 9.2 — Public-EC2 launched successfully.

10. Step 9 — Launch the Private EC2 Instance

This instance is the one we are protecting. It goes into the private subnet with **no public IP address at all**, so it is unreachable from the internet.

10.1 Launch a Second Instance

Click path: Instances → Launch instances again.

Field	What to select / type
Name	Type `Private-EC2`
AMI	**Amazon Linux 2023 AMI**
Instance type	**t3.micro**
Key pair	`test` — the **same key pair** as Public-EC2

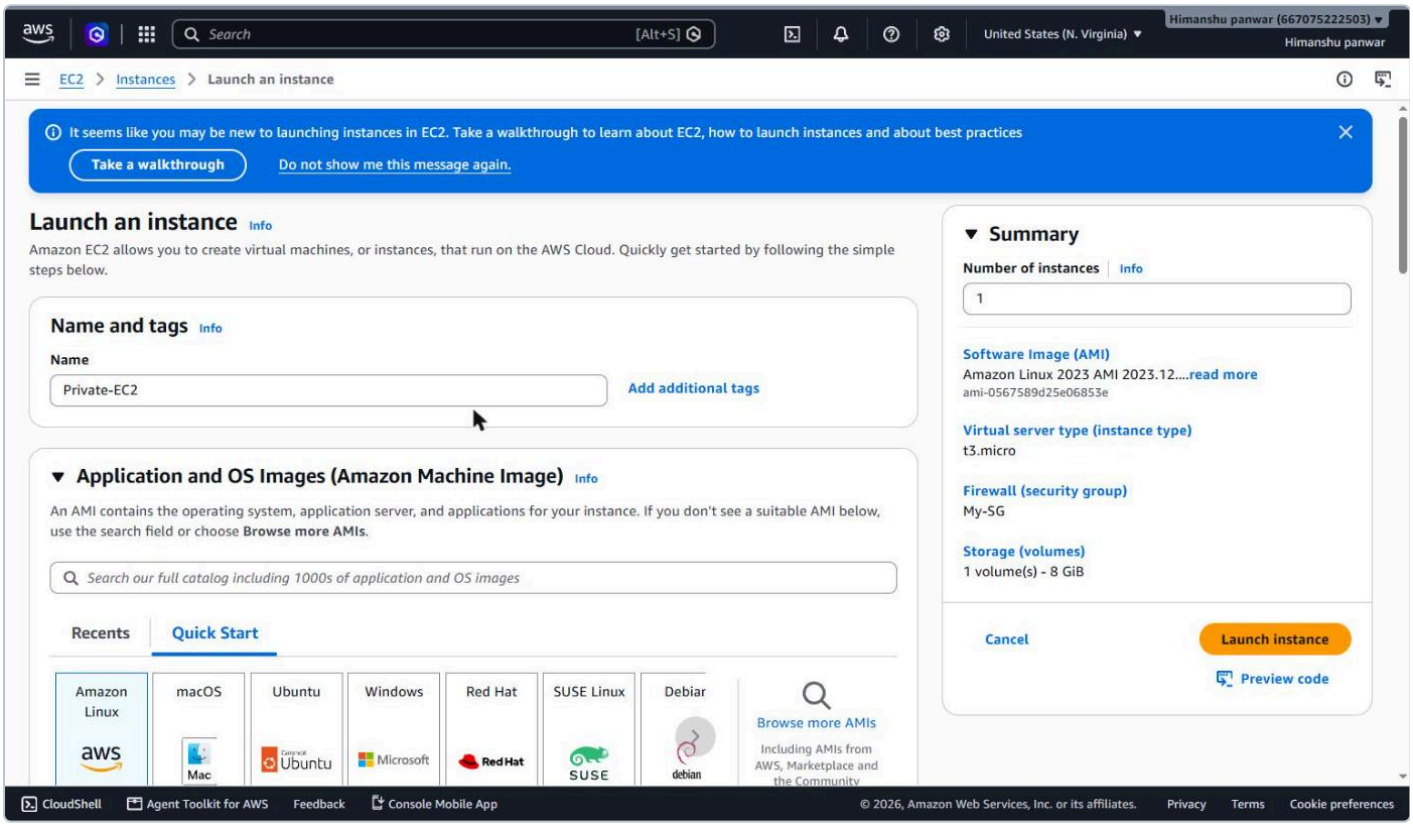


Figure 10.1 — Name it Private-EC2 and use the same AMI and key pair.

Use the same key pair

Both instances must use the **same key pair**. In Step 12 you will copy `test.pem` onto the public instance and use it to hop into the private instance. Different key pairs would break that chain.

10.2 Configure Network Settings

Click path: Network settings → Edit.

Field	What to select
VPC	Select **My-VPC-SK**
Subnet	Select **Private-Subnet** (10.0.2.0/24)
Auto-assign public IP	Set to **Disable** ← must be Disable
Firewall (security groups)	**Select existing security group** → tick **My-SG**

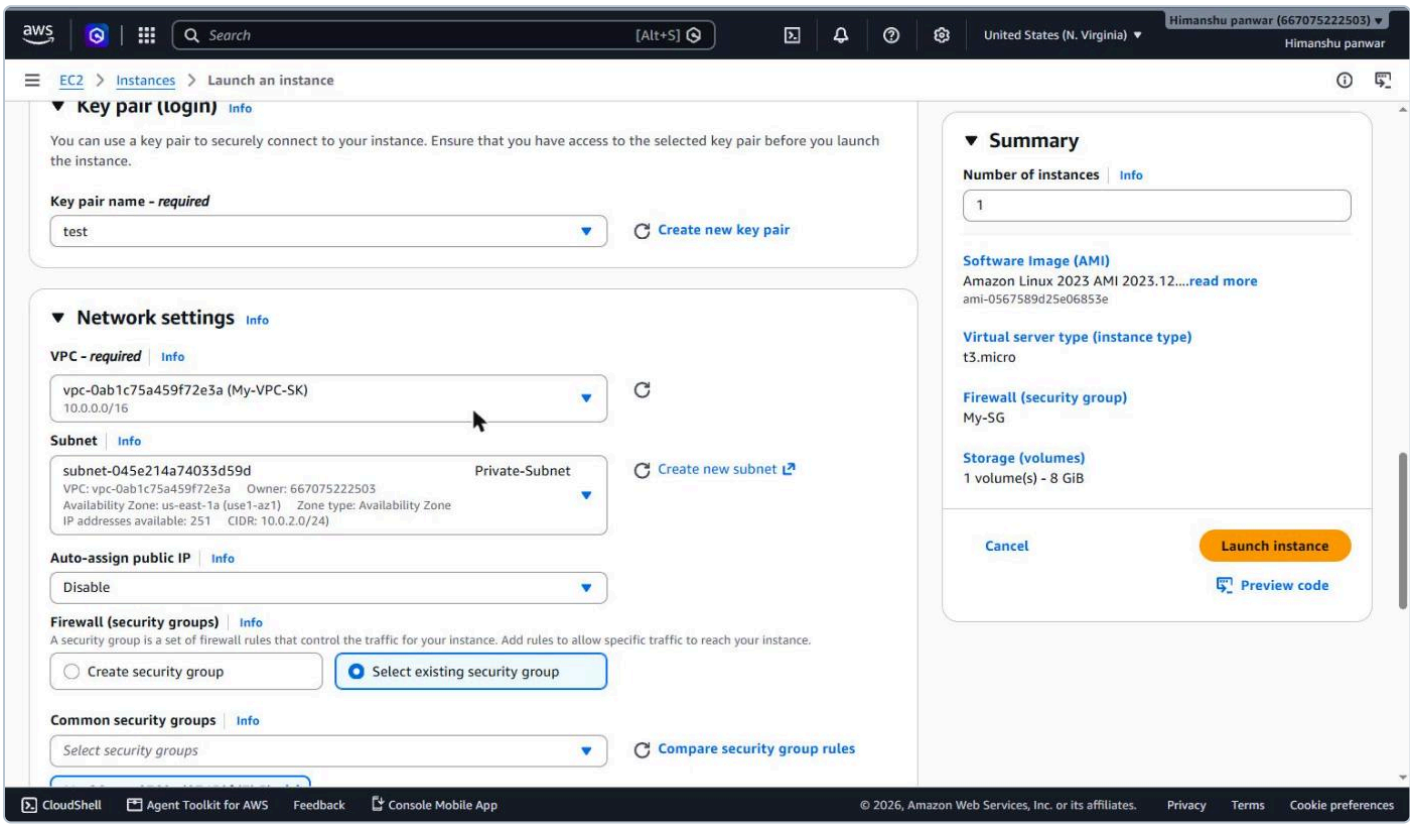


*Figure 10.2 — Private-Subnet selected and Auto-assign public IP set to **Disable**.*

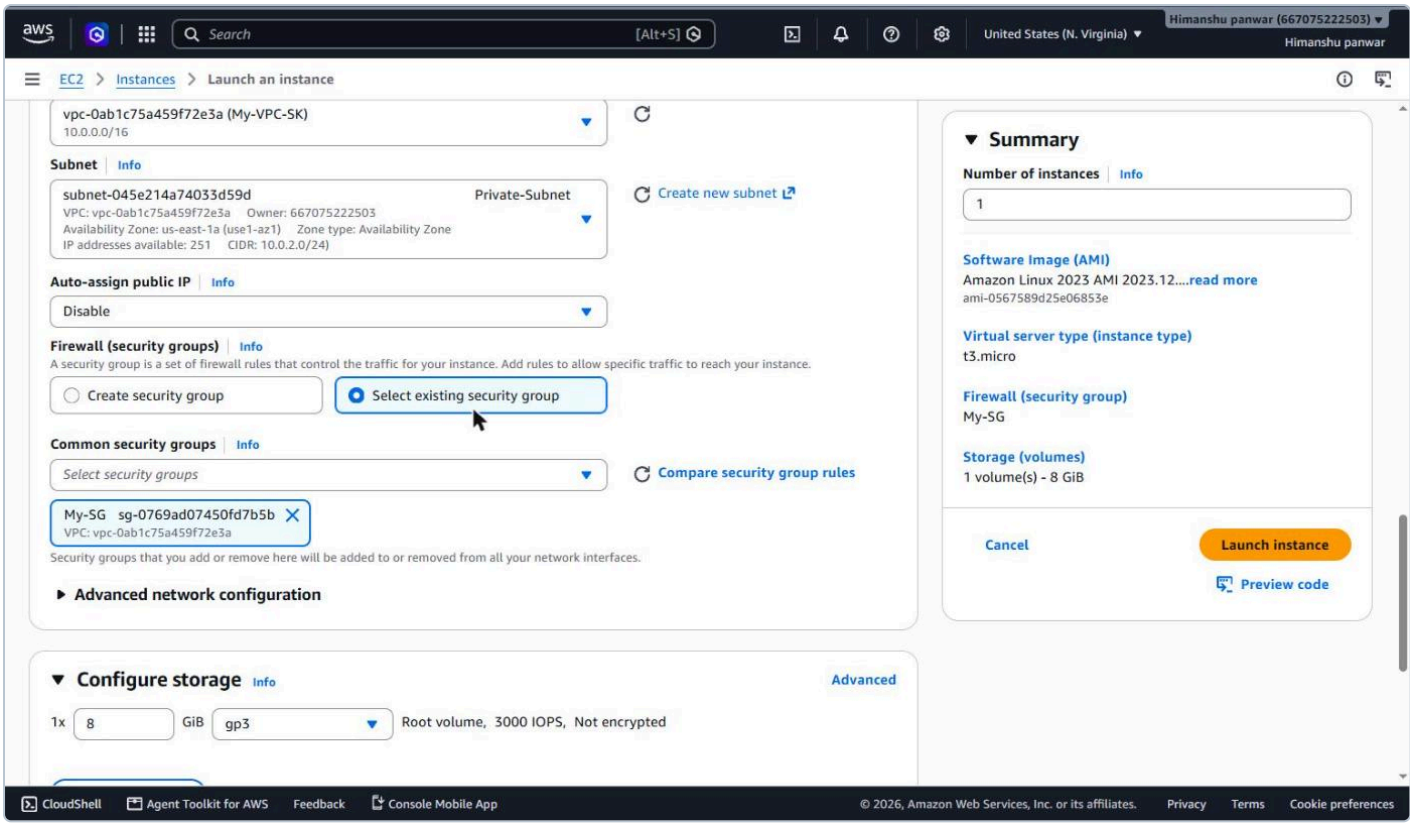


Figure 10.3 — My-SG selected as the existing security group.

10.3 Verify Both Instances

Click path: Click **Launch instance** → then go to **Instances** in the left sidebar.

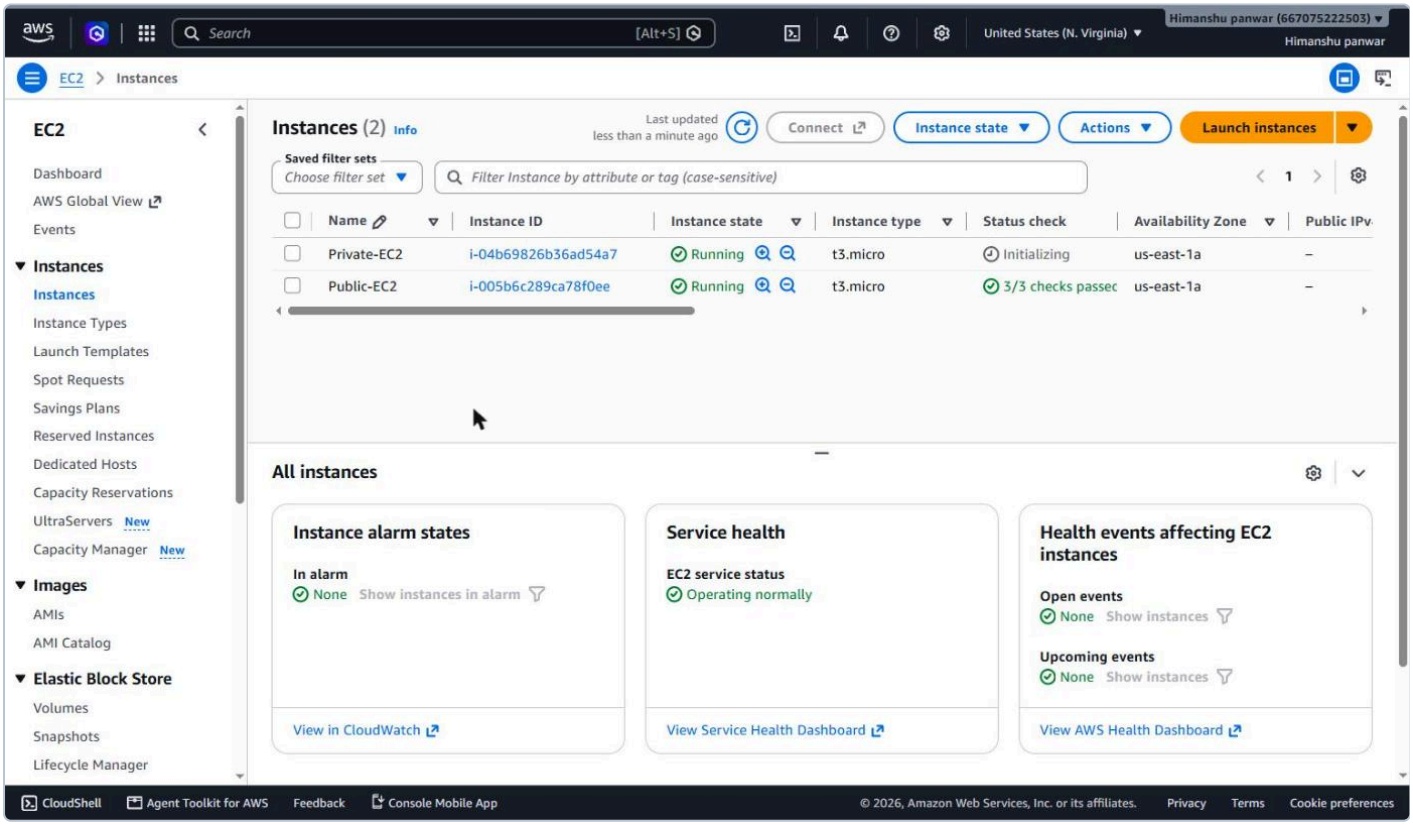


Figure 10.4 — Both instances Running. Public-EC2 has a Public IPv4 address; Private-EC2's Public IPv4 column shows a dash.

Checkpoint — record these values

- **Public-EC2** — Public IP: `13.220.8.140` · Private IP: `10.0.1.15`
- **Private-EC2** — Public IP: *none* · Private IP: `10.0.2.86`

Your IP addresses will differ. Note yours down — every command below uses them.

11. Step 10 — Test 1: SSH into the Public Instance

The first test proves that the public subnet, the Internet Gateway, the route table, and the security group are all working together correctly.

11.1 Set Key Permissions

SSH refuses to use a private key that other users on the machine can read. Run this once, from the folder that contains your `test.pem` file:

```
chmod 400 test.pem
```

> **Windows users**
> In PowerShell, use `icacls test.pem /inheritance:r /grant:r "$($env:USERNAME):(R)"` instead, or simply use Git Bash / WSL where `chmod` works normally.

11.2 Connect

```
ssh -i test.pem ec2-user@13.220.8.140
```

Replace `13.220.8.140` with your own Public-EC2 public IP. When asked "*Are you sure you want to continue connecting?*", type `yes` and press Enter.

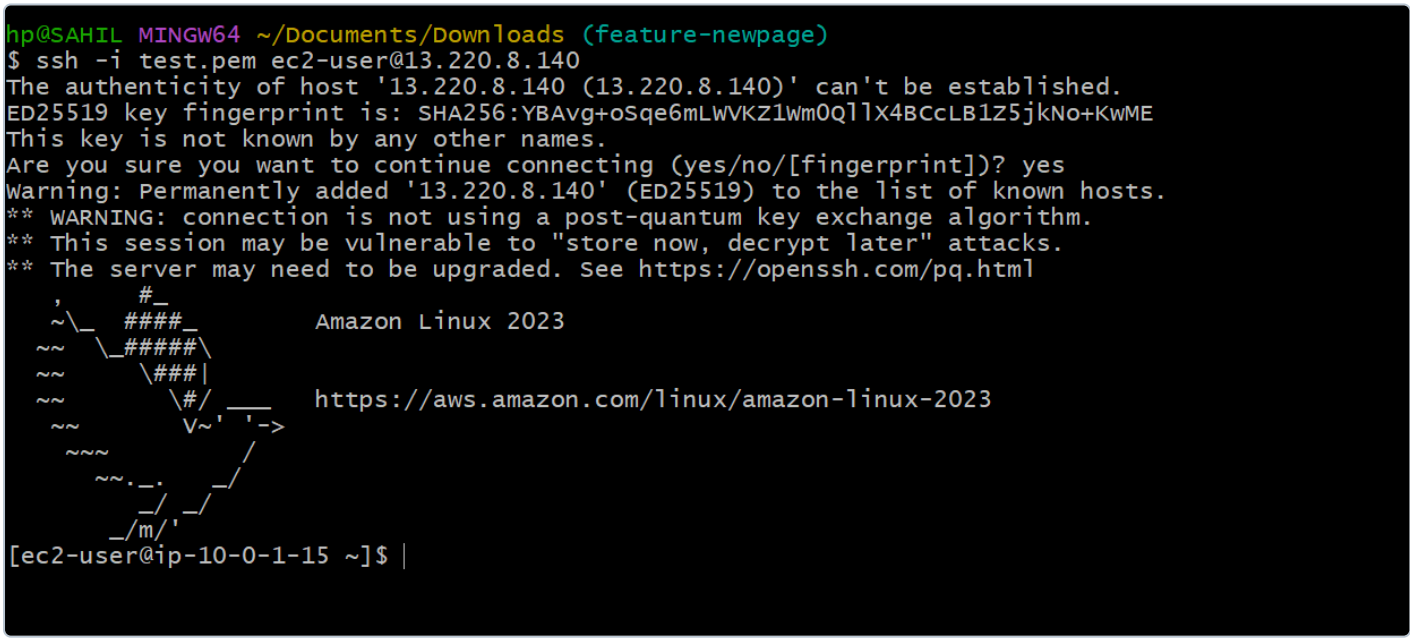


Figure 11.1 — Success. The Amazon Linux 2023 banner appears and the prompt reads `[ec2-user@ip-10-0-1-15 ~]$`.

> **What this proves**
> Your SSH packet travelled from your laptop → Internet Gateway → Public-RT (`0.0.0.0/0 → IGW`) → Public-Subnet → My-SG inbound rule port 22 → Public-EC2. Every single component in that chain is correct.

11.3 Confirm the Public Instance Has Internet

```
curl -m 10 google.com
ping -c 4 google.com
```

Both should respond immediately. (If `ping` times out but `curl` works, that is normal — ICMP is not allowed by My-SG inbound rules; outbound ping from the instance still works, so this test should succeed.)

12. Step 11 — Test 2: Prove the Private Instance Is Isolated

Before building the jump server, confirm that the private instance really is unreachable from the outside.

12.1 Try to SSH Directly from Your Laptop

From your **laptop** (not from Public-EC2), run:

```
ssh -i test.pem ec2-user@10.0.2.86
```

Expected result: the command hangs and eventually fails with `Operation timed out` or `No route to host`.

> **Why this fails — and why that is correct**
>
> `10.0.2.86` is a **private** RFC 1918 address. It is not routable on the public internet; no router outside your VPC knows where to send it. Private-EC2 also has no public IP, so there is no public address to connect to in the first place. **This failure is the whole point of a private subnet.**

13. Step 12 — Test 3: The Bastion Host (Jump Server)

A **Bastion Host** (also called a Jump Server) is a hardened instance in the public subnet that acts as the single, auditable entry point into your private network. You SSH into the bastion, then SSH onward from there.

```
Your Laptop              Public-EC2               Private-EC2
(internet)       →       10.0.1.15         →       10.0.2.86
                       (bastion / jump)          (protected server)

ssh -i test.pem              ssh -i ~/.ssh/test.pem
ec2-user@13.220.8.140        ec2-user@10.0.2.86
```

13.1 Copy the Key to the Bastion

Private-EC2 requires the same key to authenticate, so the key must be present on Public-EC2. From your **laptop**, run:

```
scp -i test.pem test.pem ec2-user@13.220.8.140:~/.ssh/
```

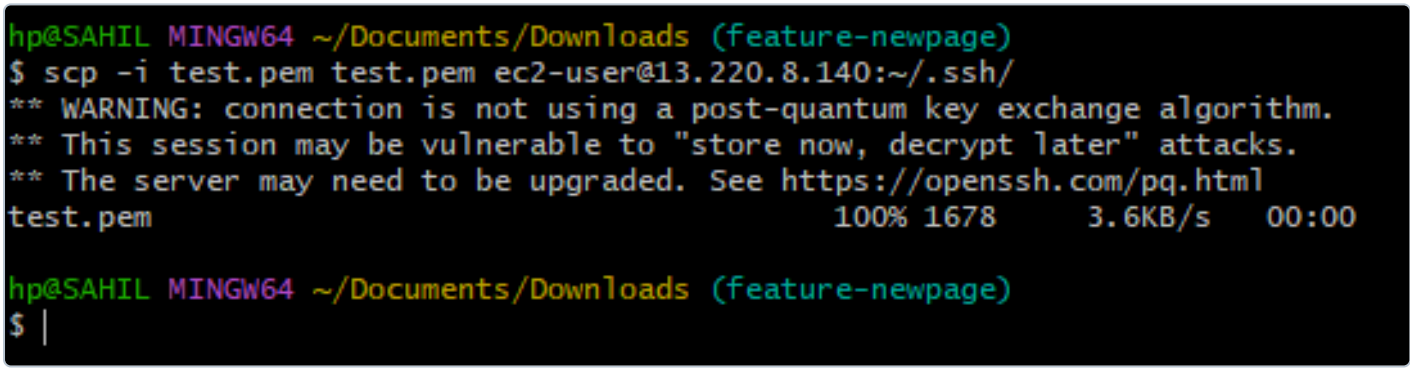


Figure 13.1 — `test.pem` copied to `~/.ssh/` on the bastion; transfer shows 100%.

> **Security note for real environments**
>
> Copying a private key onto a bastion is fine for a lab, but in production you should use **SSH agent forwarding** (`ssh -A`) or **AWS Systems Manager Session Manager** instead. That way the key never leaves your laptop.

13.2 Set Key Permissions on the Bastion

SSH into Public-EC2 and fix the permissions of the copied key:

```
ssh -i test.pem ec2-user@13.220.8.140
chmod 400 ~/.ssh/test.pem
```

13.3 Jump to the Private Instance

Now, **from inside the Public-EC2 shell**, run:

```
ssh -i ~/.ssh/test.pem ec2-user@10.0.2.86
```

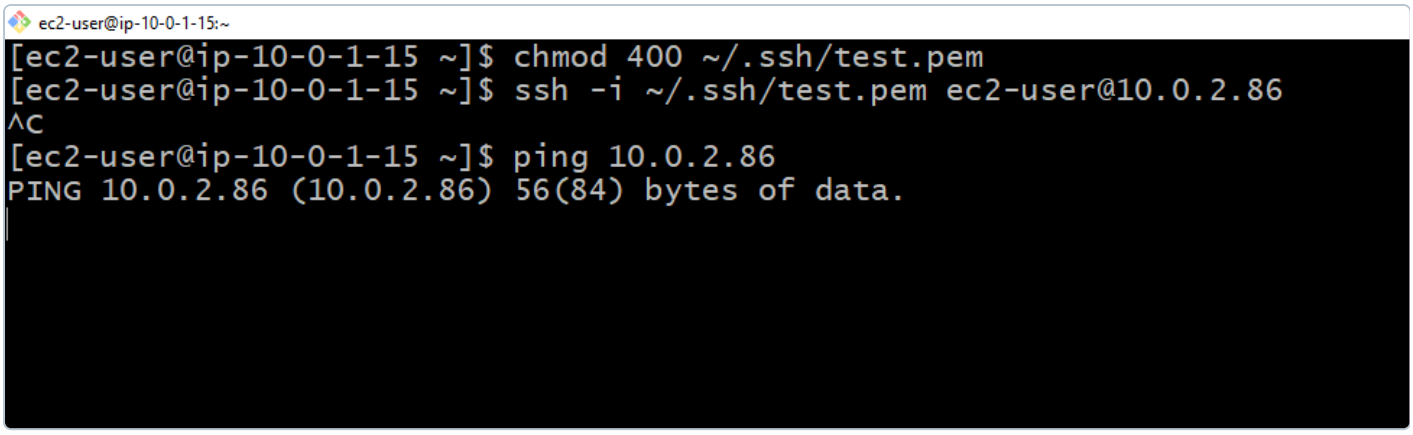


Figure 13.2 — The jump command issued from the bastion shell.

If this hangs instead of connecting

During this lab, this exact command hung with no error and no timeout. The root cause was a **missing outbound rule on My-SG**. If you hit the same wall, go to **Section 17: Troubleshooting** now — it walks through the full diagnosis using AWS VPC Reachability Analyzer and the one-line fix.

13.4 Success

When it works, the prompt changes from `ip-10-0-1-15` to `ip-10-0-2-86`:

```
[ec2-user@ip-10-0-1-15 ~]$ ssh -i ~/.ssh/test.pem ec2-user@10.0.2.86
Warning: Permanently added '10.0.2.86' (ED25519) to the list of known hosts.
[ec2-user@ip-10-0-2-86 ~]$
```

What this proves

The `10.0.0.0/16 → local` route lets the two subnets talk to each other inside the VPC, and My-SG allows port 22 between them. You now have controlled access to a server that has **zero exposure to the public internet**.

14. Step 13 — Test 4: Prove the Private Instance Has No Internet

This test demonstrates *why* a NAT Gateway is needed. Right now Private-RT has only the local route — there is no path out of the VPC.

14.1 Run the Test

From inside the **Private-EC2** shell (prompt shows `ip-10-0-2-86`), run:

```
curl -m 5 google.com
```

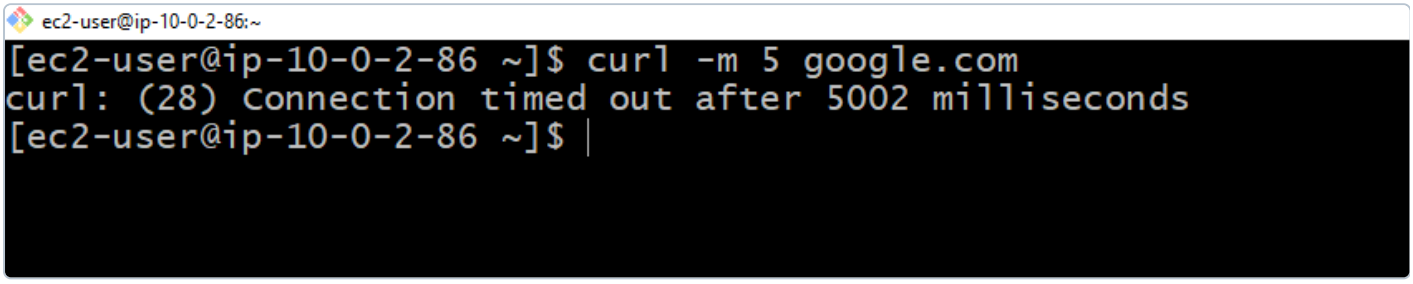


Figure 14.1 — `curl: (28) Connection timed out after 5002 milliseconds` — exactly the expected failure.

Why it fails

Private-RT contains only `10.0.0.0/16 → local`. When the instance tries to reach `142.250.x.x` (Google), the VPC router finds no matching route and silently drops the packet. There is no error message because nothing is rejecting it — the packet simply has nowhere to go.

Real-world impact

Without outbound internet, the private instance cannot run `yum update`, install packages, pull Docker images, call external APIs, or send logs to a third-party service. That is the problem a NAT Gateway solves.

15. Step 14 — Create the NAT Gateway

A **NAT (Network Address Translation) Gateway** lets instances in a private subnet initiate outbound connections to the internet, while blocking all unsolicited inbound connections. It is a one-way door.

	Internet Gateway	NAT Gateway
Traffic direction	Inbound and outbound	Outbound only
Lives in	The VPC itself	A **public** subnet
Needs a public IP	No	Yes — an Elastic IP
Used by	Public subnets	Private subnets
Cost	Free	Hourly + per-GB charges

15.1 Start the Creation Wizard

Click path: Go to the **VPC** console → left sidebar **NAT gateways** → click **Create NAT gateway**.

15.2 Fill in the Settings

Field	What to select / type
Name	Type `My-NAT-GW`
Availability mode	Select **Zonal** ← see the note below
Subnet	Select **Public-Subnet** ← must be the PUBLIC subnet
Connectivity type	Select **Public**
Elastic IP allocation ID	Click the **Allocate Elastic IP** button on the right

Gotcha — the Subnet field is hidden by default

AWS now defaults the **Availability mode** to **Regional**, and in Regional mode AWS places the gateway for you — so **the Subnet dropdown does not appear at all**. You must click **Zonal** first; the Subnet field then appears immediately below.

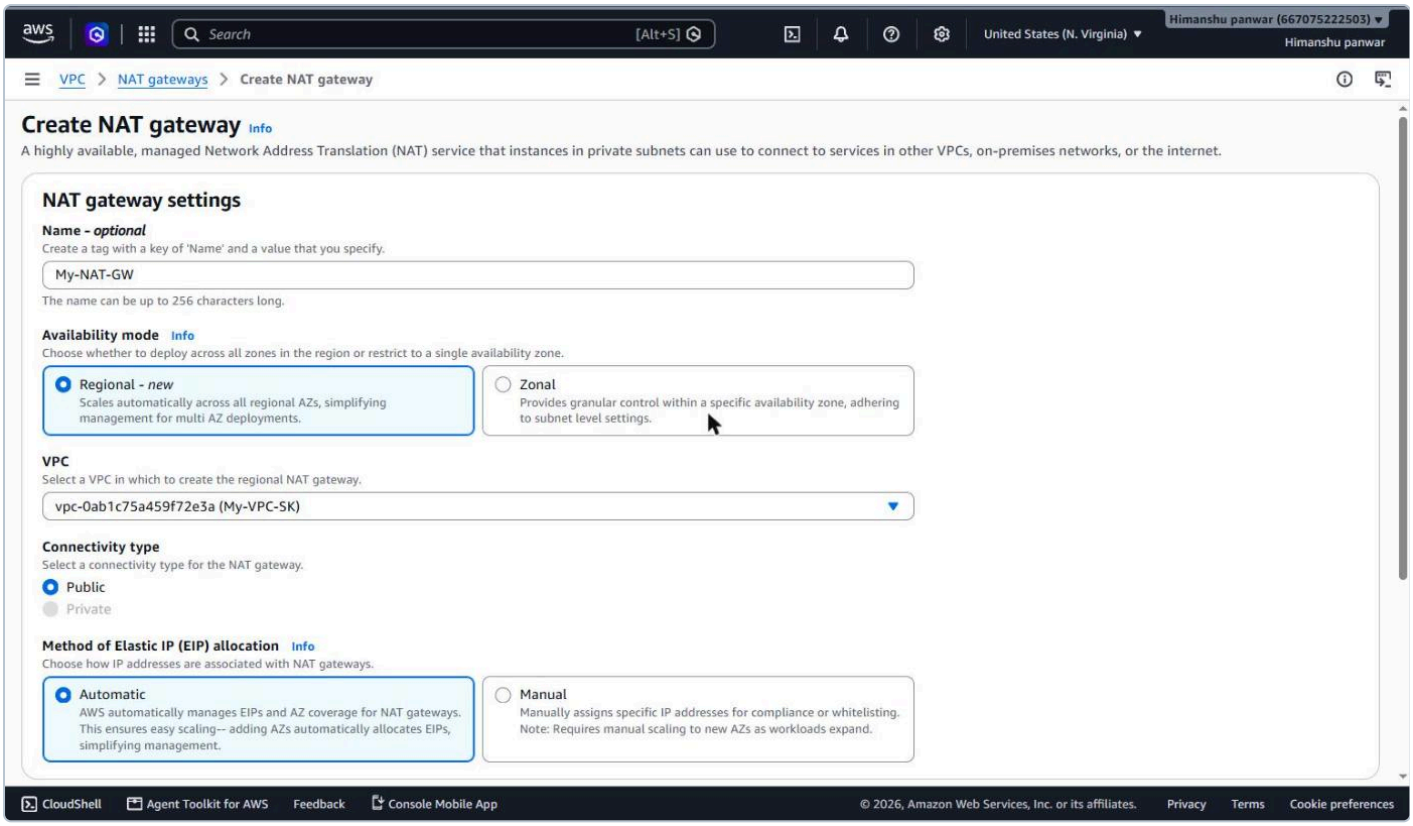


*Figure 15.1 — The Create NAT gateway form in Regional mode. Note there is no Subnet field — switch to **Zonal** to reveal it.*

> **Common mistake**
>
> Placing the NAT Gateway in the **private** subnet. A NAT Gateway must sit in a subnet whose route table points to an Internet Gateway — otherwise it has no way to reach the internet itself, and nothing works. **Always select Public-Subnet.**

15.3 Create and Wait

Click path: Scroll down → click **Create NAT gateway**.

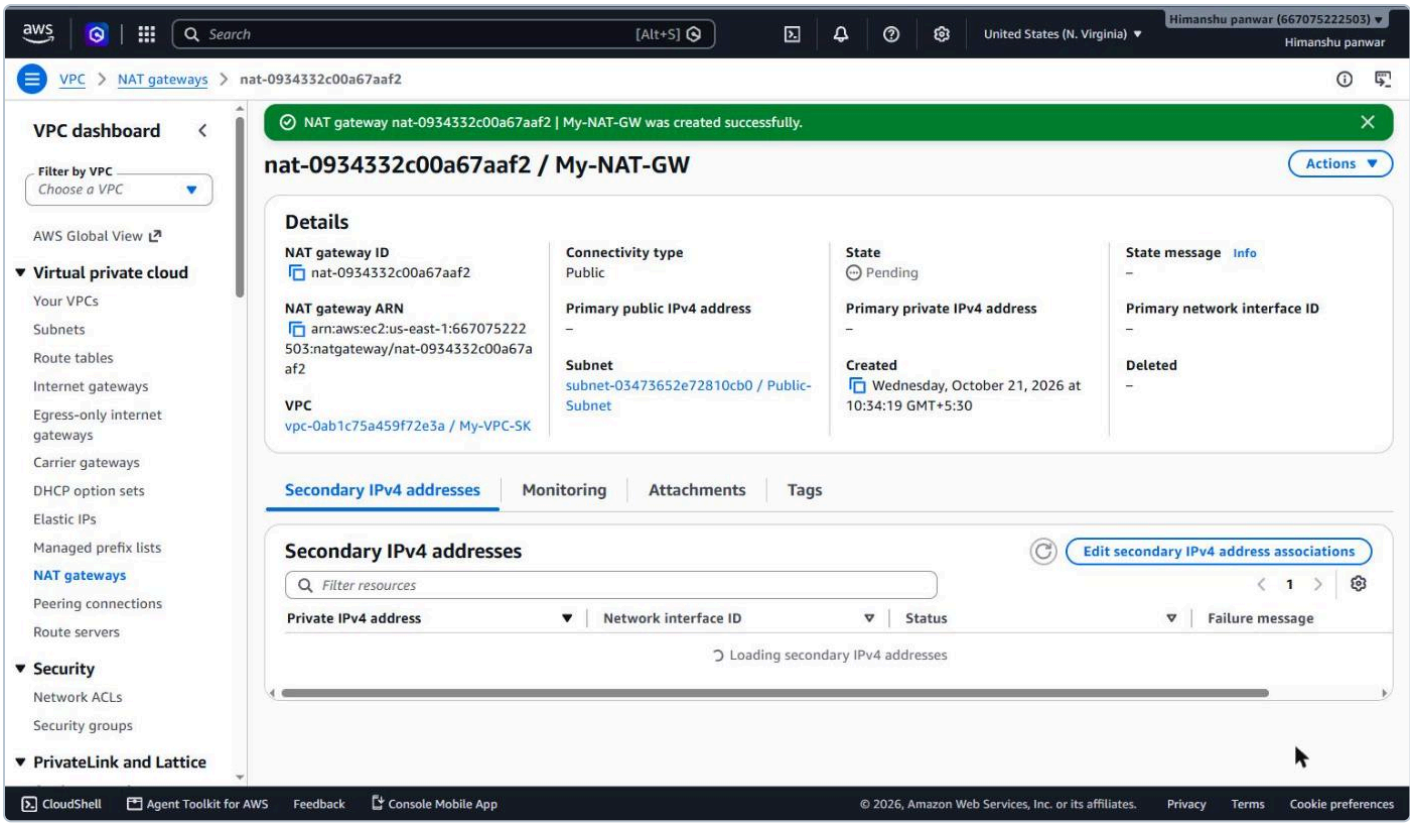


*Figure 15.2 — NAT gateway created. State shows **Pending**; it becomes **Available** in about 2–3 minutes.*

Checkpoint

Wait until **State = Available** before continuing. Refresh the page with the circular arrow button. In this lab the gateway received Elastic IP `34.195.54.3` — this is the public IP the whole private subnet will appear to come from.

16. Step 15 — Route Private Traffic Through the NAT Gateway

The NAT Gateway exists, but Private-EC2 still does not know to use it. Add a default route to Private-RT.

16.1 Edit Private-RT

Click path: VPC console → **Route tables** → select **Private-RT** → **Routes** tab → **Edit routes** → **Add route**.

Field	What to select / type
Destination	Type `0.0.0.0/0`
Target	Click the dropdown → select **NAT Gateway** → select **My-NAT-GW**

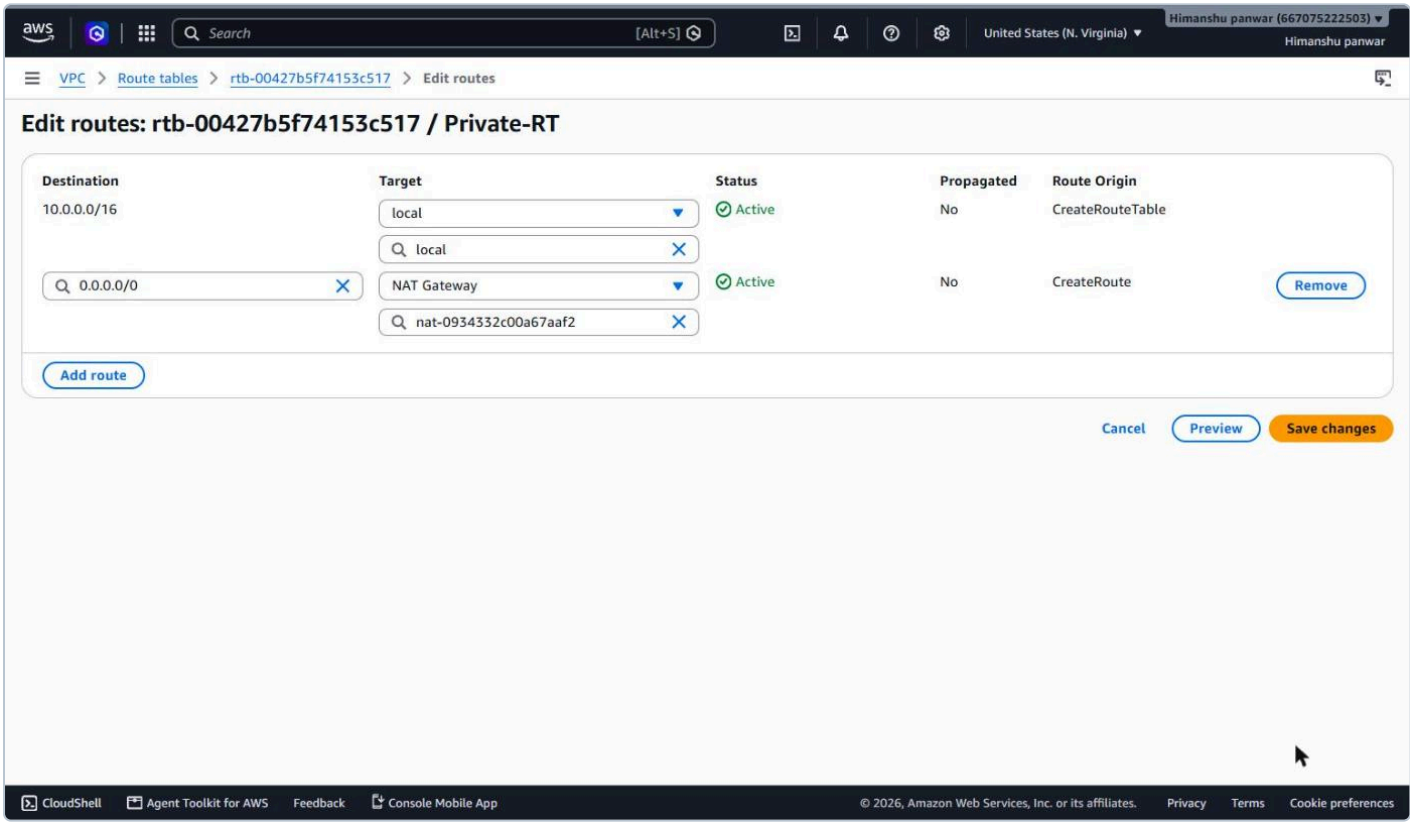


Figure 16.1 — Private-RT now has two routes: `10.0.0.0/16 → local` and `0.0.0.0/0 → nat-0934332c00a67aaf2`, both Active.

Click path: Click **Save changes**.

16.2 Final Route Table Comparison

	Public-RT	Private-RT
Local route	`10.0.0.0/16 → local`	`10.0.0.0/16 → local`
Default route	`0.0.0.0/0 → igw-xxxx`	`0.0.0.0/0 → nat-xxxx`
Inbound from internet	**Allowed**	**Blocked**
Outbound to internet	**Allowed**	**Allowed (via NAT)**

16.3 Final Test — Private Instance Reaches the Internet

SSH back into Private-EC2 through the bastion and run the same commands as before:

```
ssh -i test.pem ec2-user@13.220.8.140
ssh -i ~/.ssh/test.pem ec2-user@10.0.2.86
curl -m 10 google.com
ping google.com
```

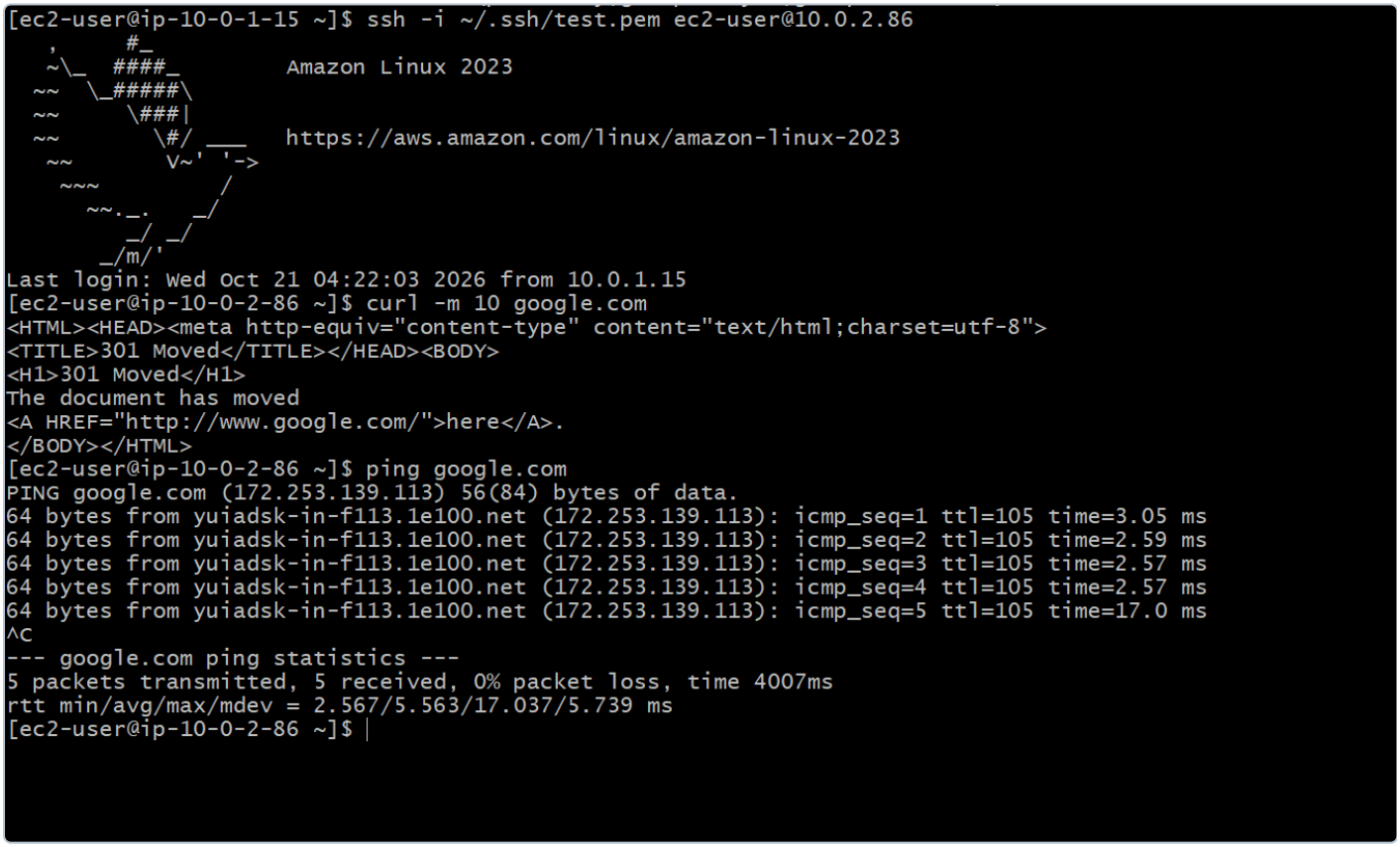


*Figure 16.2 — Success. `curl` returns Google's **301 Moved** HTML and `ping` shows **0% packet loss** — all through the NAT Gateway.*

Lab complete — what you just proved

- Private-EC2 can now **reach out** to the internet (package updates, API calls, Docker pulls all work).
- The internet still **cannot reach in** — Private-EC2 has no public IP and the NAT Gateway accepts no inbound connections.
- To the outside world, all traffic from the private subnet appears to come from the Elastic IP `34.195.54.3` .

17. Troubleshooting — The Missing Outbound Rule

This section documents a real problem encountered during this lab

Everything above was configured correctly — routes, subnets, Internet Gateway, inbound security group rules — and yet SSH from the bastion to the private instance silently hung. This is the full diagnostic walkthrough. The technique shown here (VPC Reachability Analyzer) works for *any* "traffic is not flowing and I don't know why" problem in AWS.

17.1 The Symptom

At Step 12 (Section 13.3), this command from the bastion produced no output and never timed out:

```
[ec2-user@ip-10-0-1-15 ~]$ ssh -i ~/.ssh/test.pem ec2-user@10.0.2.86
<cursor just sits there, forever>
```

17.2 Step 1 of Diagnosis — Run SSH in Verbose Mode

The first move with any hanging SSH is to add `-v` (verbose) so SSH tells you how far it got:

```
ssh -v -i ~/.ssh/test.pem ec2-user@10.0.2.86
```

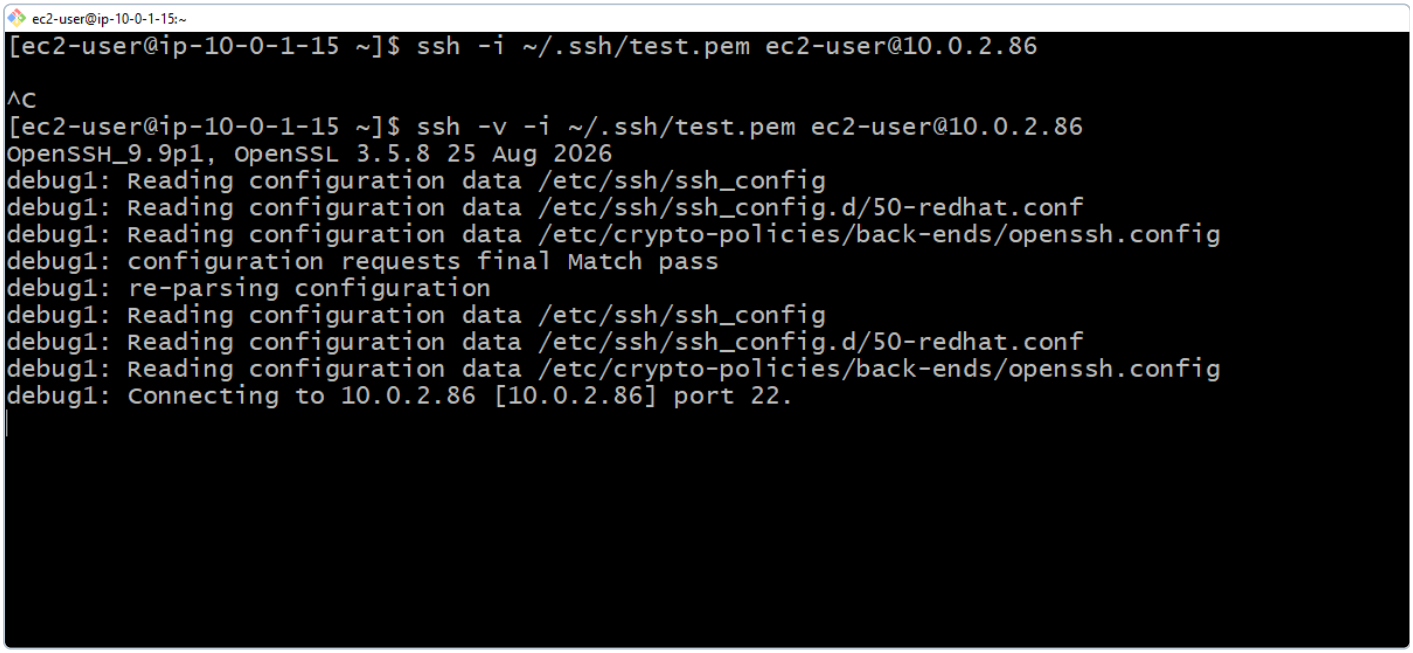


Figure 17.1 — Verbose output stops at `debug1: Connecting to 10.0.2.86 [10.0.2.86] port 22.` and goes no further.

Reading this output — the single most useful clue

SSH never printed a banner, never negotiated a key exchange, never mentioned authentication. It stopped at **"Connecting..."**, which means the **TCP handshake itself never completed**. The SYN packet left the bastion and no SYN-ACK came back.

That rules out everything at the SSH layer — wrong key, wrong username, wrong permissions, sshd not running. The problem is purely at the **network layer**: a security group, a NACL, or a route.

17.3 Step 2 — Rule Out the Obvious Suspects

Four things were checked manually, and all four were correct:

Suspect	What was checked	Result
Instance health	EC2 → Private-EC2 → Status checks	**3/3 passed** — the OS is up and healthy
SG inbound rules	My-SG → Inbound rules tab	**Correct** — SSH port 22 from 0.0.0.0/0 present
Network ACL	Private-Subnet → Network ACL tab	**Correct** — default NACL, allows all in and out
Route table	Private-RT → Routes tab	**Correct** — `10.0.0.0/16 → local` covers 10.0.1.x → 10.0.2.x

At this point the obvious causes were exhausted. Rather than keep guessing, the right move is to let AWS tell you where the packet dies.

17.4 Step 3 — Use VPC Reachability Analyzer

VPC Reachability Analyzer traces a hypothetical packet through every hop of your network configuration and names the exact component that blocks it. It is a configuration analysis, not a live ping — the instances do not even need to be running.

Click path: AWS Console search bar → type `Reachability Analyzer` → click it → click **Create and analyze path**.

Field	What to select
Name tag	Type anything, e.g. `bastion-to-private`
Source type	Select **Instance**
Source	Select **Public-EC2** (`i-005b6c289ca78f0ee`)
Destination type	Select **Instance**
Destination	Select **Private-EC2** (`i-04b69826b36ad54a7`)
Protocol	Select **TCP**
Destination port	Type `22`

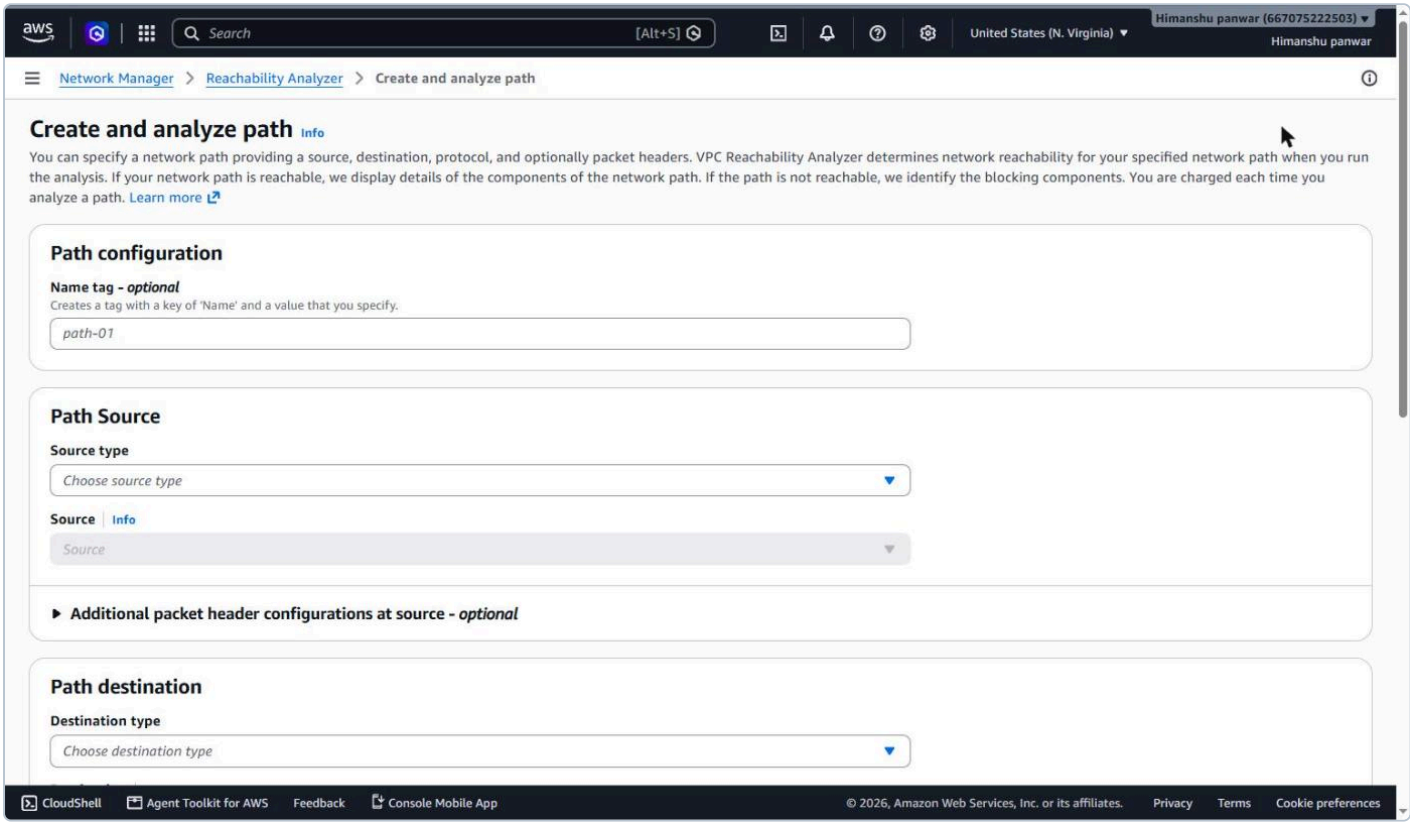


Figure 17.2 — The Create and analyze path form. Source = bastion, Destination = private instance, TCP port 22.

Click path: Click **Create and analyze path** and wait about 30 seconds for the analysis to finish.

17.5 Step 4 — Read the Verdict

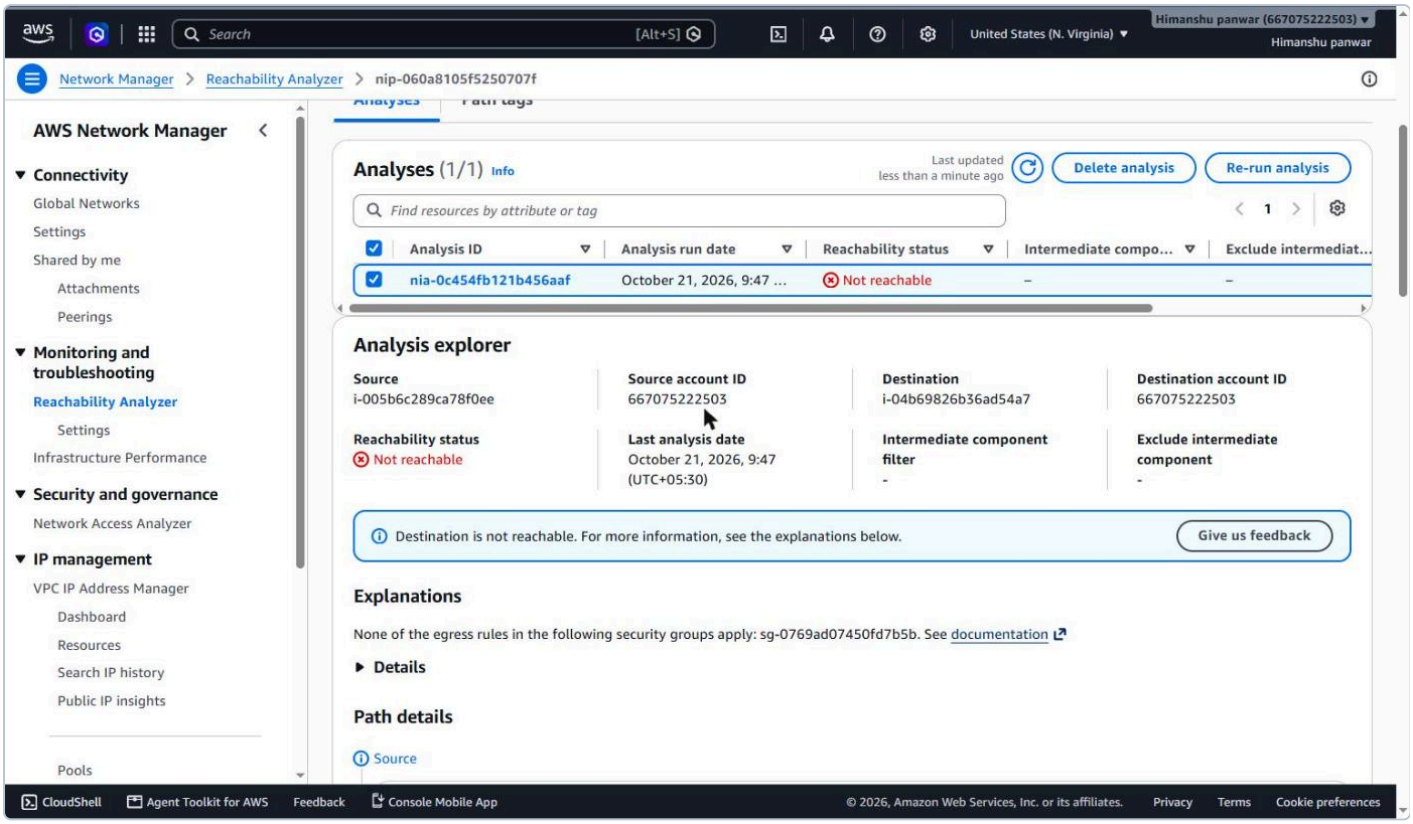


*Figure 17.3 — Reachability status: **Not reachable**. The Explanations section names the exact cause.*

The **Explanations** section gave the answer in one sentence:

```
None of the egress rules in the following security groups apply:
sg-0769ad07450fd7b5b
```

Root cause

My-SG had zero outbound (egress) rules. The Security group detail page confirmed it: **Outbound rules count: 0 Permission entries**.

With an empty egress list, the security group blocks *every* packet leaving the instance. The bastion's SSH SYN packet was dropped the moment it tried to exit Public-EC2's network interface — it never even reached the private subnet. Because the packet was silently dropped rather than rejected, SSH received no error and simply waited forever. That is exactly the behaviour seen in Figure 17.1.

17.6 Why Did This Happen?

The subtle AWS behaviour behind this bug

When you create a security group through the AWS Console, it normally comes with a default outbound rule allowing **All traffic to 0.0.0.0/0**. However, if during creation you interact with the **Outbound rules** section and remove that default row — or if the console is left in a state where no outbound row is present when you click Create — the group is created with an **empty egress list**.

This is easy to miss because:

- The console shows no warning or error.
- Inbound rules look perfect, so attention goes there first.
- Security groups are **stateful**, so *incoming* SSH to the bastion kept working — the reply to an allowed inbound connection is always permitted regardless of egress rules. Only **newly initiated outbound** connections were blocked.

That last point is the real trap: the bastion was reachable *from* the laptop, which made the security group look healthy, while it could not initiate a connection *to* anything.

17.7 The Fix

Click path: EC2 console → **Security groups** → select **My-SG** → **Outbound rules** tab → click **Edit outbound rules** → click **Add rule**.

Field	Value to set
Type	Select **All traffic**
Protocol	Auto-fills to **All** (greyed out)
Port range	Auto-fills to **All** (greyed out)
Destination type	Select **Anywhere-IPv4**
Destination	`0.0.0.0/0`

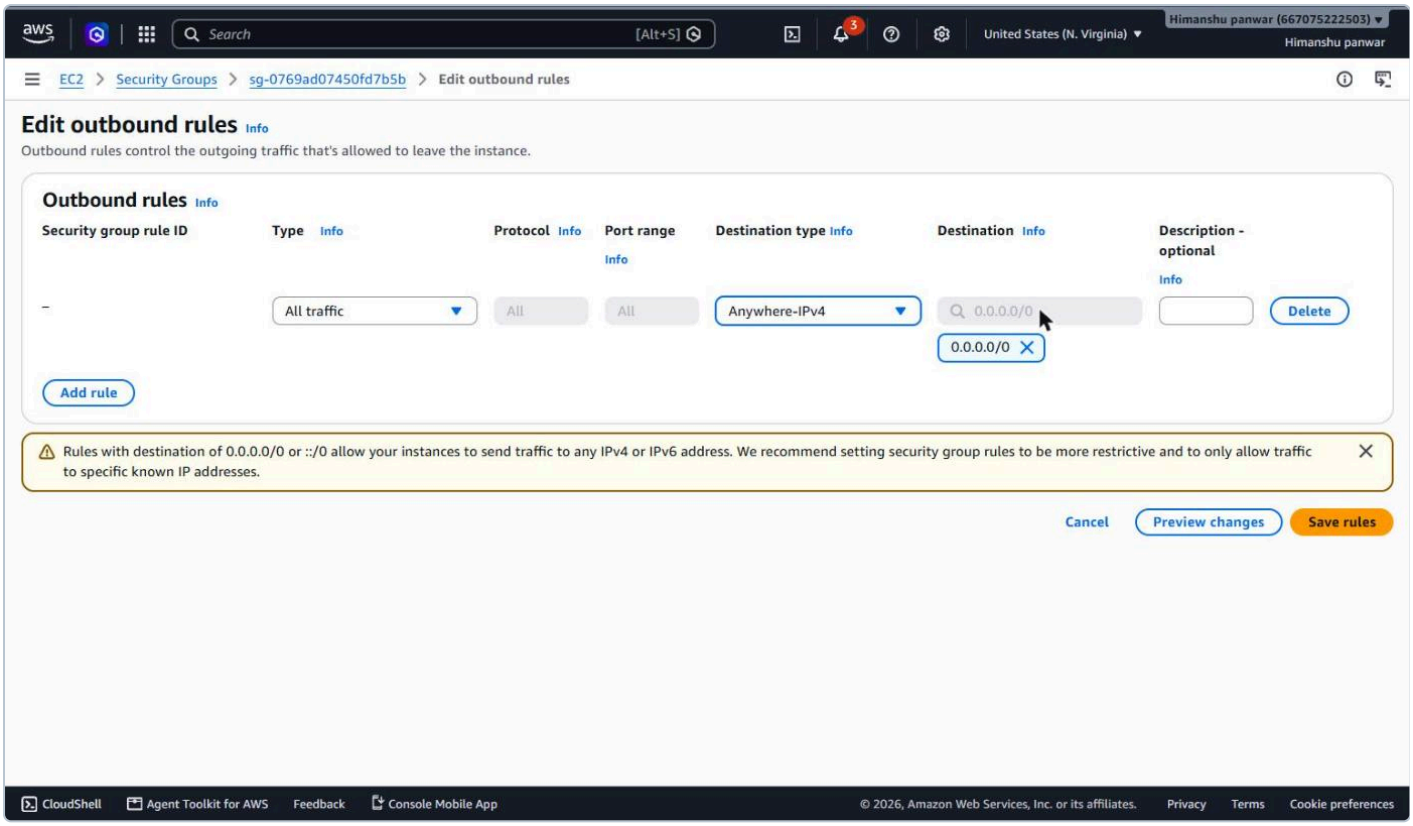


Figure 17.4 — Adding the outbound rule: All traffic → Anywhere-IPv4 (0.0.0.0/0). The orange banner is a standard advisory, not an error.

Click path: Click **Save rules**.

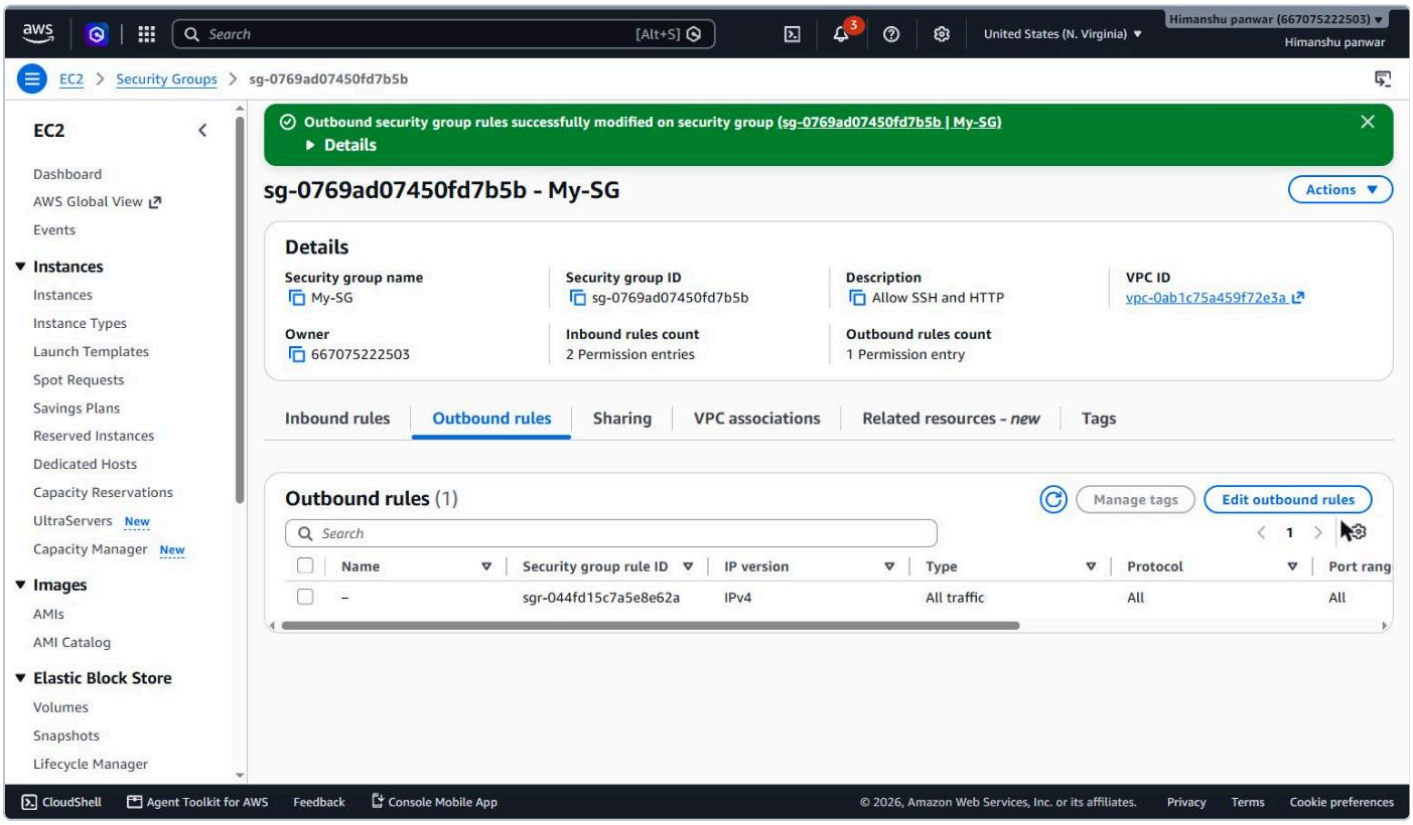


*Figure 17.5 — Green banner confirms the change. **Outbound rules count** now reads **1 Permission entry**.*

17.8 Verification

Security group changes take effect **immediately** — no instance restart is needed. The jump command was retried straight away:

```
[ec2-user@ip-10-0-1-15 ~]$ ssh -i ~/.ssh/test.pem ec2-user@10.0.2.86
Warning: Permanently added '10.0.2.86' (ED25519) to the list of known hosts.
[ec2-user@ip-10-0-2-86 ~]$
```

Connection established instantly. Problem solved.

17.9 Lessons to Carry Forward

Lesson	Detail
Always verify egress	After creating any security group, open the Outbound rules tab and confirm the count is not 0. Inbound rules get all the attention; outbound rules fail silently.
Hang ≠ reject	A security group **drops** blocked packets without sending a rejection. That is why you get an infinite hang instead of "Connection refused". A refused connection usually means the service is down; a hang usually means a firewall or routing problem.
Use `ssh -v` first	Where the verbose output stops tells you which layer is broken. Stopping at "Connecting..." = network layer. Getting to "Authentications that can continue" = key or permissions problem.
Stateful cuts both ways	Statefulness can mask a broken egress rule, because reply traffic to an allowed inbound connection is always permitted. Test an *outbound-initiated* connection to be sure.
Stop guessing, start analyzing	VPC Reachability Analyzer named the exact security group ID in one run, after four manual checks found nothing. Reach for it early — it costs a few cents per analysis and saves hours.

18. Cleanup — Delete Resources to Avoid Charges

Do this as soon as you finish

The NAT Gateway and Elastic IP bill by the hour whether you use them or not. Delete resources in the order below — AWS refuses to delete a resource that something else still depends on.

Order	Resource	Click path
1	EC2 Instances	EC2 → Instances → select both → **Instance state** → **Terminate instance**
2	NAT Gateway	VPC → NAT gateways → select My-NAT-GW → **Actions** → **Delete NAT gateway** (takes a few minutes)
3	Elastic IP	VPC → Elastic IPs → select the IP → **Actions** → **Release Elastic IP addresses**
4	Internet Gateway	VPC → Internet gateways → **Actions** → **Detach from VPC**, then **Actions** → **Delete**
5	Subnets	VPC → Subnets → select both → **Actions** → **Delete subnet**
6	Route Tables	VPC → Route tables → select Public-RT and Private-RT → **Actions** → **Delete**
7	Security Group	EC2 → Security groups → select My-SG → **Actions** → **Delete security groups**
8	VPC	VPC → Your VPCs → select My-VPC-SK → **Actions** → **Delete VPC**

Shortcut

After terminating the instances and deleting the NAT Gateway, you can often select the VPC and choose **Delete VPC** — AWS will offer to delete the subnets, route tables, Internet Gateway, and non-default security groups along with it. You still need to release the Elastic IP separately.

19. Lab Summary

19.1 Completion Checklist

#	Task	Result
1	Create VPC (My-VPC-SK, 10.0.0.0/16)	DONE
2	Create Public-Subnet (10.0.1.0/24)	DONE
3	Create Private-Subnet (10.0.2.0/24)	DONE
4	Create and attach Internet Gateway	DONE
5	Create Public-RT with route to IGW	DONE
6	Create Private-RT (local route only)	DONE
7	Create Security Group (SSH + HTTP)	DONE
8	Launch Public-EC2 with public IP	DONE
9	Launch Private-EC2 without public IP	DONE
10	Test 1 — SSH into Public-EC2 from laptop	PASS
11	Test 2 — Direct SSH to Private-EC2 blocked	PASS
12	Test 3 — Bastion host jump to Private-EC2	PASS
13	Diagnose and fix missing SG egress rule	FIXED
14	Test 4 — Private-EC2 has no internet (pre-NAT)	PASS
15	Create NAT Gateway with Elastic IP	DONE
16	Add 0.0.0.0/0 → NAT route to Private-RT	DONE
17	Test 5 — Private-EC2 reaches internet via NAT	PASS

19.2 Key Concepts Mastered

Concept	What you learned
CIDR blocks	How `/16` and `/24` define address space, and the five IPs AWS reserves in every subnet.
Public vs private subnet	The difference is *entirely* in the route table — there is no "make public" checkbox.
Internet Gateway	Two-way door at the VPC level; useless until a route points at it.
Route tables	Most-specific route wins; `local` is automatic and immutable; `0.0.0.0/0` is the default route.
Security Groups	Stateful, instance-level, allow-rules only. **Both inbound and outbound must be checked.**
Network ACLs	Stateless, subnet-level, supports deny rules — the second layer of defence.
Bastion Host	A single hardened entry point into a private network, replacing public IPs on every server.
NAT Gateway	One-way outbound door for private subnets; must live in a public subnet and needs an Elastic IP.
Reachability Analyzer	Traces a packet through your config and names the blocking component — the fastest path from symptom to root cause.

19.3 Quick Command Reference

```
# Set private key permissions (run once, on your laptop)
chmod 400 test.pem

# SSH into the bastion (public instance)
ssh -i test.pem ec2-user@<PUBLIC-EC2-PUBLIC-IP>

# Copy the key onto the bastion so it can jump onward
scp -i test.pem test.pem ec2-user@<PUBLIC-EC2-PUBLIC-IP>:~/.ssh/

# On the bastion: fix key permissions
chmod 400 ~/.ssh/test.pem

# On the bastion: jump to the private instance
ssh -i ~/.ssh/test.pem ec2-user@<PRIVATE-EC2-PRIVATE-IP>

# Test internet access from any instance
curl -m 10 google.com
ping -c 4 google.com

# Diagnose a hanging SSH connection
ssh -v -i <key> ec2-user@<ip>

# One-liner jump without copying the key (ProxyJump — preferred in production)
ssh -i test.pem -J ec2-user@<PUBLIC-IP> ec2-user@<PRIVATE-IP>
```

End of Lab

You have built a complete, correctly segmented AWS network and verified every security boundary with a real test — including diagnosing and fixing a genuine production-style bug. This same architecture (public subnet for load balancers and bastions, private subnet for application servers and databases, NAT Gateway for outbound updates) is the standard foundation of nearly every AWS workload.